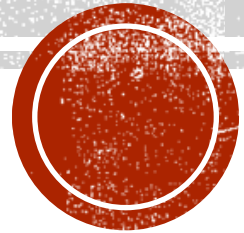
1. FOUNDATIONS OF NEURAL NETWORKS AND DEEP LEARNING



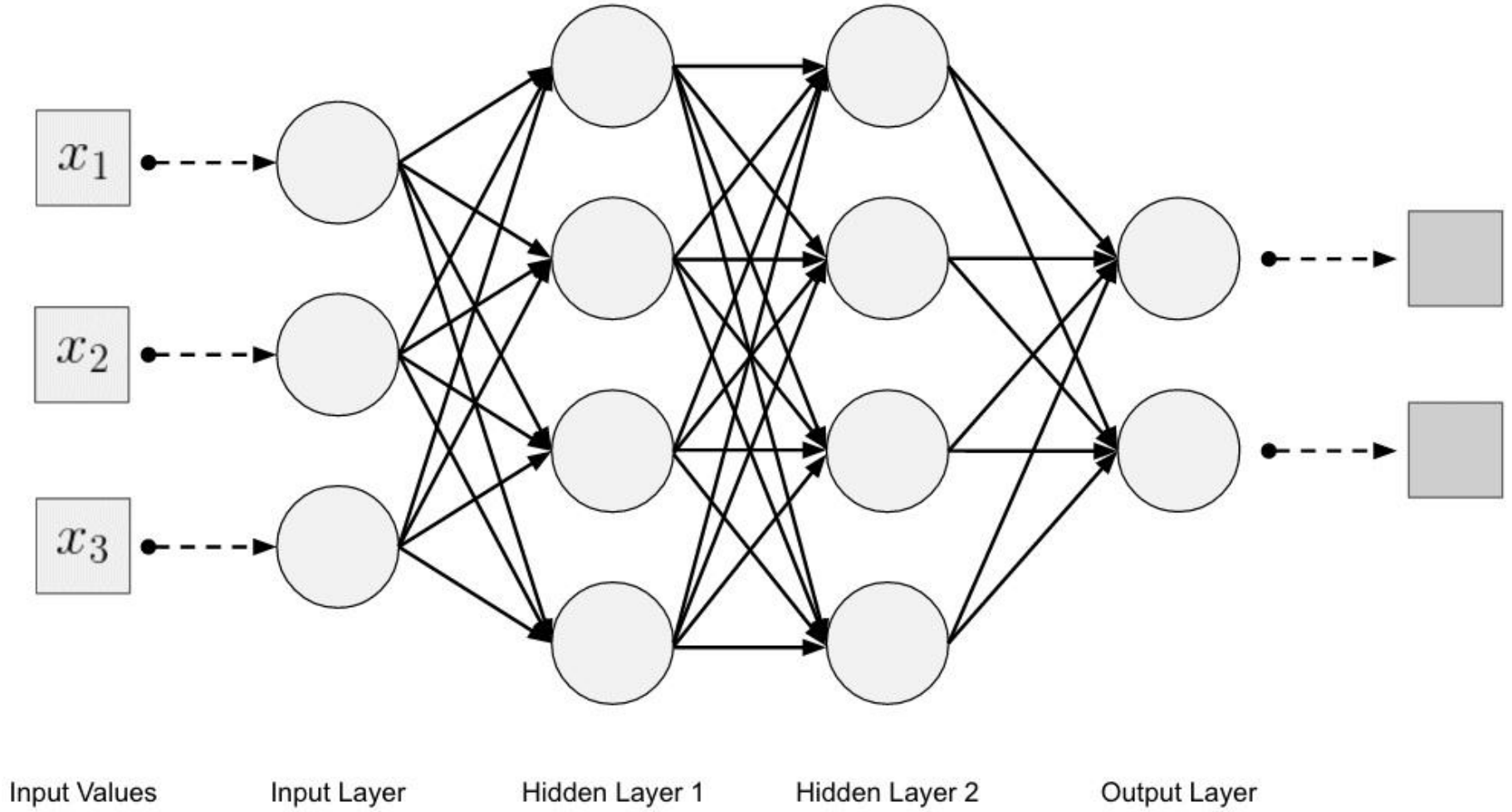
NEURAL NETWORKS

The behavior of neural networks is shaped by its network architecture

- Number of neurons
- Number of layers
- Types of connections between layers



MULTILAYER NEURAL NETWORK TOPOLOGY



Multilayer neural network topology

MULTILAYER NEURAL NETWORK TOPOLOGY

- *Connection weights*-coefficients that scale (amplify or minimize) the input signal to a given neuron in the network
- *Biases*-
  - Scalar values added to the input to ensure that at least few nodes per layer are activated regardless of signal strength.
  - Biases allow learning to happen by giving the network action in the event of low signal.
  - Allow the network to try new interpretations or behaviors.
  - Notated as *b*, and, like weights, biases are modified throughout the learning process
- *Activation functions*-The functions that govern the artificial neuron's behavior



PARAMETRIC VS. NONPARAMETRIC LEARNING

- Parametric models- use fixed parameters, assuming data distribution (e.g., linear regression),
  - **Supervised parametric learning** learns from labeled data with fixed assumptions (Logistic Regression)
  - **unsupervised parametric learning** finds patterns in unlabeled data using fixed forms (like k-means with fixed k).
- Nonparametric models- adapt to data with flexible parameters (e.g., KNN, Decision Trees).
  - **Nonparametric learning** (both supervised & unsupervised) makes fewer assumptions, handles complex data better, but needs more data.
- **Transfer Learning** leverages pre-trained models to solve new, related problems, saving time and data.



PARAMETRIC MODELS

- Parametric models-
  - Make strong assumptions about the functional form, or shape, of the relationship between the variables in the data.
  - Characterized by having a fixed number of parameters, which are estimated from the training data and used to make predictions.



PARAMETRIC MODELS

- **Key Features of Parametric Models**
  - **Assumptions:** assume a specific form for the relationship between variables (e.g., linear regression assumes a linear relationship).
  - **Parameterized:** Fixed model parameters and are typically estimated using techniques like maximum likelihood estimation or least squares.
  - **Efficiency:** Often computationally efficient and require fewer training data to estimate parameters compared to non-parametric models.
  - **Interpretability:** Parameters often have straightforward interpretations, making them useful for inference and understanding the relationship between variables.



NON-PARAMETRIC MODELS

- Make fewer assumptions about the functional form of the relationship between variables.
- Instead of a fixed set of parameters, these models typically have parameters that grow with the size of the data, allowing them to be more flexible in modeling complex relationships.



NON-PARAMETRIC MODELS

- **Key Features of Non-Parametric Models**
  - **Flexibility: D**o not assume a specific functional form, making them versatile in capturing complex patterns in data.
  - **Parameterization:** The number of parameters may vary with the size of the dataset, which can lead to increased computational complexity but greater flexibility.
  - **Data Requirements:** Require larger amounts of data to estimate relationships accurately due to their flexibility.
  - **Interpretability:** They may offer less straightforward interpretations of parameters compared to parametric models, focusing more on predictive accuracy than on understanding the underlying relationships.



Aspect	Parametric Models	Non-Parametric Models
Assumptions	Strong assumptions about data distribution and relationships (e.g., linear regression assumes linearity).	Fewer assumptions about data distribution and relationships.
Flexibility	Limited flexibility due to fixed functional form.	High flexibility to model complex relationships.
Interpretability	Parameters have clear interpretations.	Focus on predictive accuracy; parameters less interpretable.



Aspect	Parametric Models	Non-Parametric Models
Computational Efficiency	Generally more computationally efficient.	Can be computationally intensive with large datasets.
Data Requirements	Less data required for parameter estimation.	More data needed for accurate modeling of complex patterns.
Robustness	Vulnerable to model misspecification.	More robust to deviations from assumptions.
Overfitting Risk	Prone to underfitting complex data patterns.	Risk of overfitting if not properly regularized.



Aspect	Parametric Models	Non-Parametric Models
Training Time	Faster training time due to simpler estimation procedures.	Slower training time, especially with large datasets.
Application Areas	Well-suited for linear relationships and well-understood data.	Ideal for complex, nonlinear relationships and unknown data distributions.



ACTIVATION FUNCTIONS

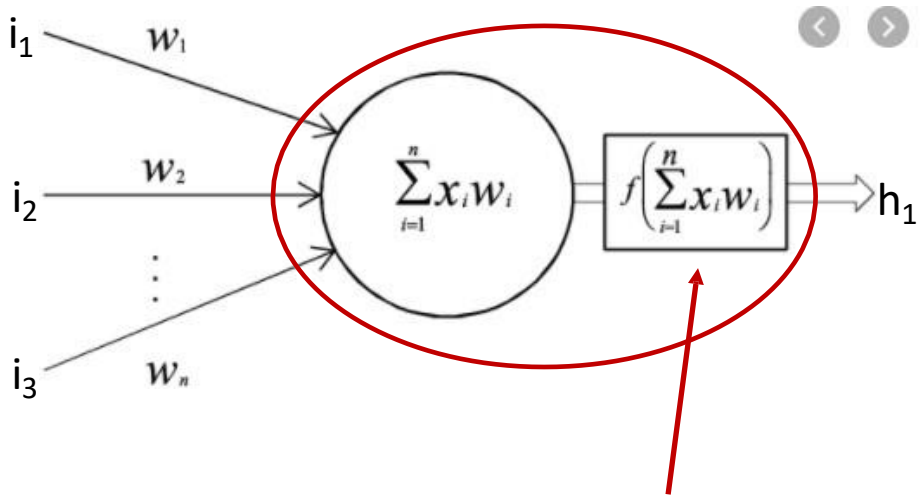
Activation functions-

- Propagate the output of one layer's nodes forward to the next layer (up to and including the output layer).
- a scalar-to-scalar function, yielding the neuron's activation.
- Used for hidden neurons in a neural network to introduce nonlinearity into the network's modeling capabilities.
- Many activation functions belong to a logistic class of transforms that (when graphed) resemble an S.
- This class of function is called *sigmoidal*.
  - The sigmoid family of functions contains several variations, one of which is known as the Sigmoid function.

https://www.101ai.net/deeplearn/activation



ACTIVATION FUNCTIONS



- Activation Function is applied over the linear weighted summation of the incoming information to a node.
- Convert linear input signals from perceptron to a linear/non-linear output signal.
- It decides whether to activate a node or not.

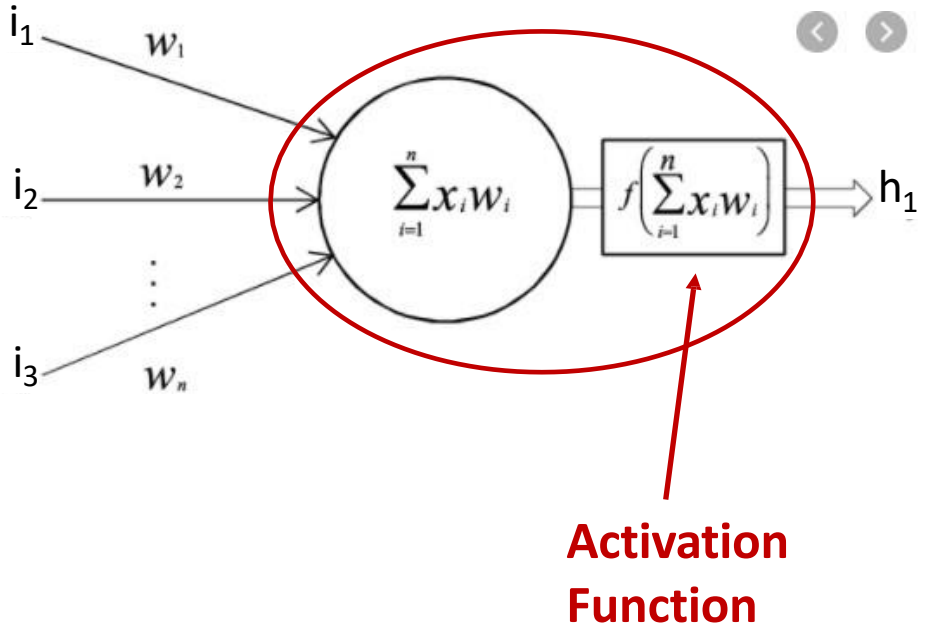


ACTIVATION FUNCTIONS

Activation functions must be monotonic, differentiable, and quickly converging.

Types of Activation Functions:

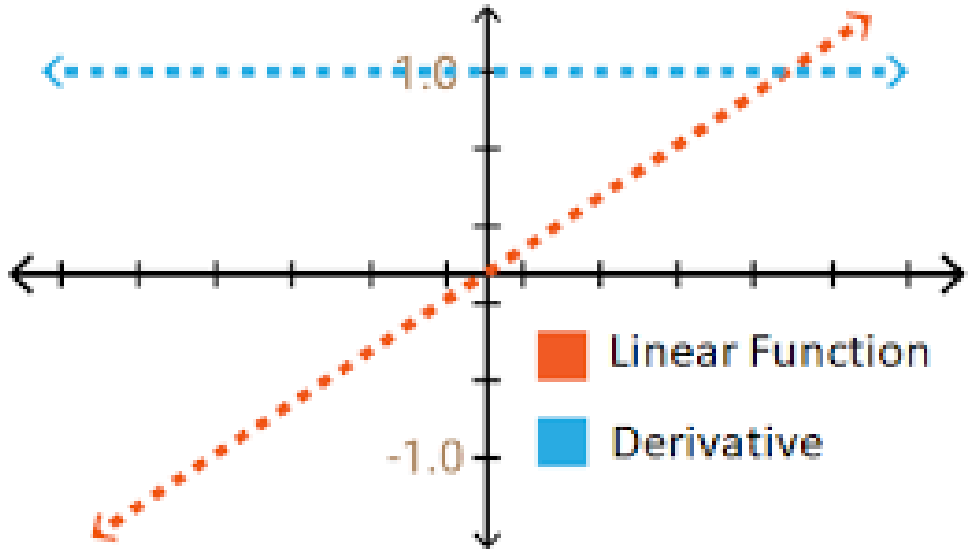
- Linear
- Non-Linear



LINEAR

$$f(x) = ax + b$$

$$\frac{df(x)}{dx} = a$$



Observations:

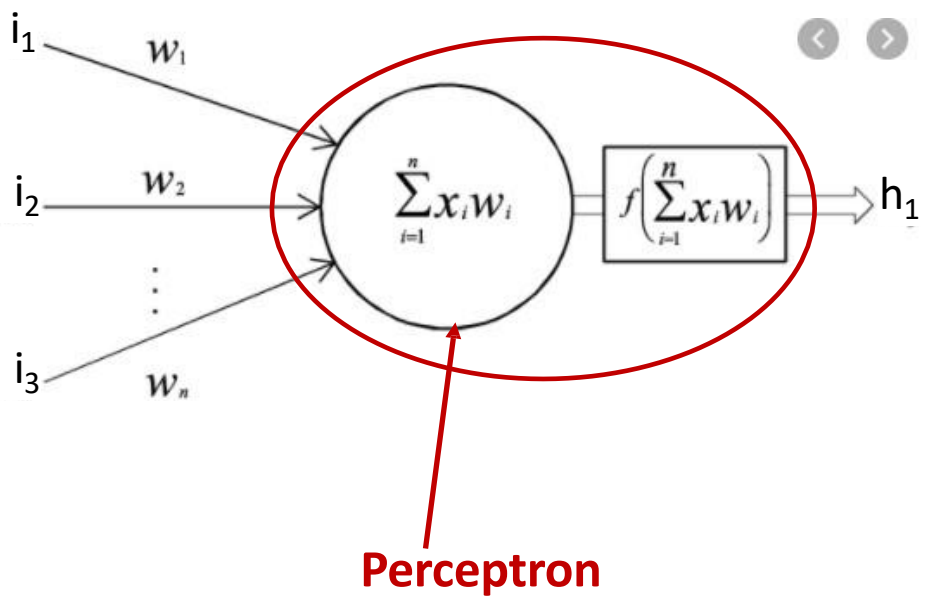
- Constant gradient
- Gradient does not depend on the change in the input



LINEAR

$$f(x) = ax + b$$

$$f(x) = a_1x_1 + a_2x_2 + a_3x_3 + \cdots + b$$

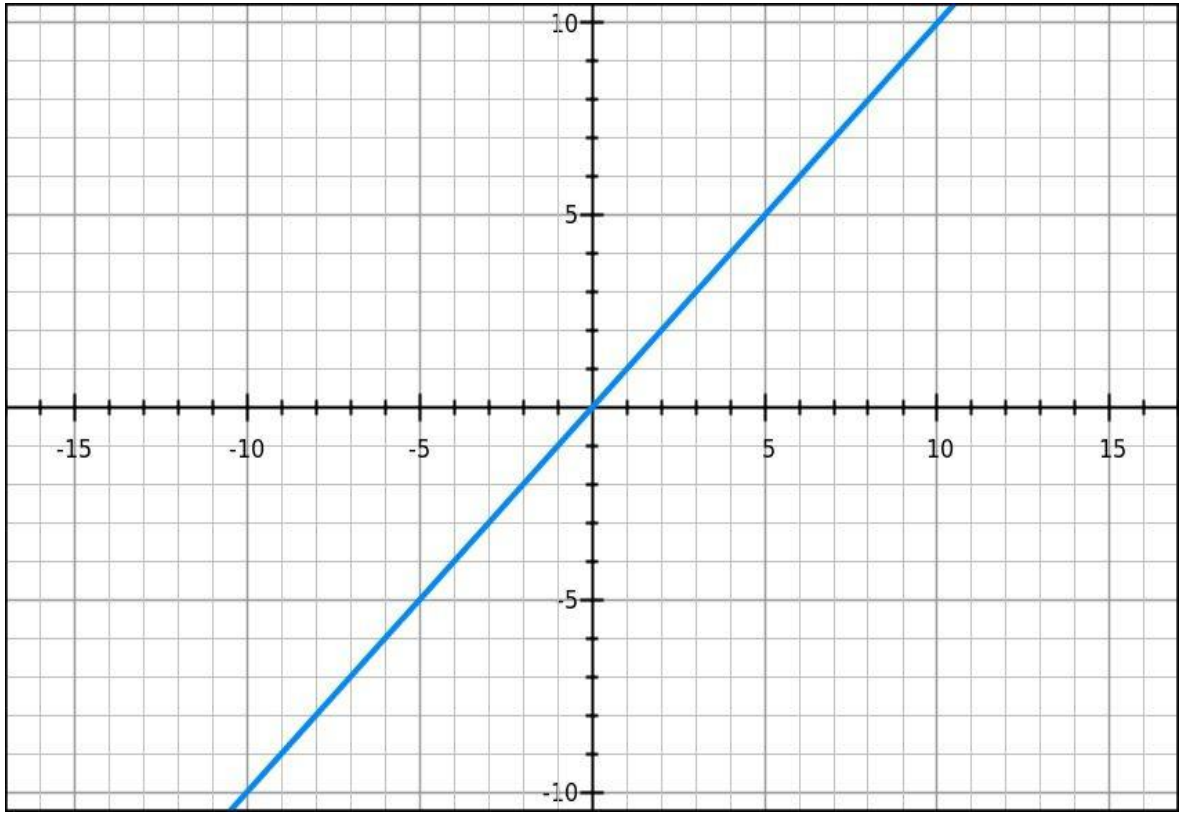


NON-LINEAR

- Sigmoid (Logistic)
- Hyperbolic Tangent (Tanh)
- Rectified Linear Unit (ReLU)
  - *Leaky Relu*
  - *Parametric Relu*
- Exponential Linear Unit (ELU)



ACTIVATION FUNCTIONS-LINEAR



Linear activation function

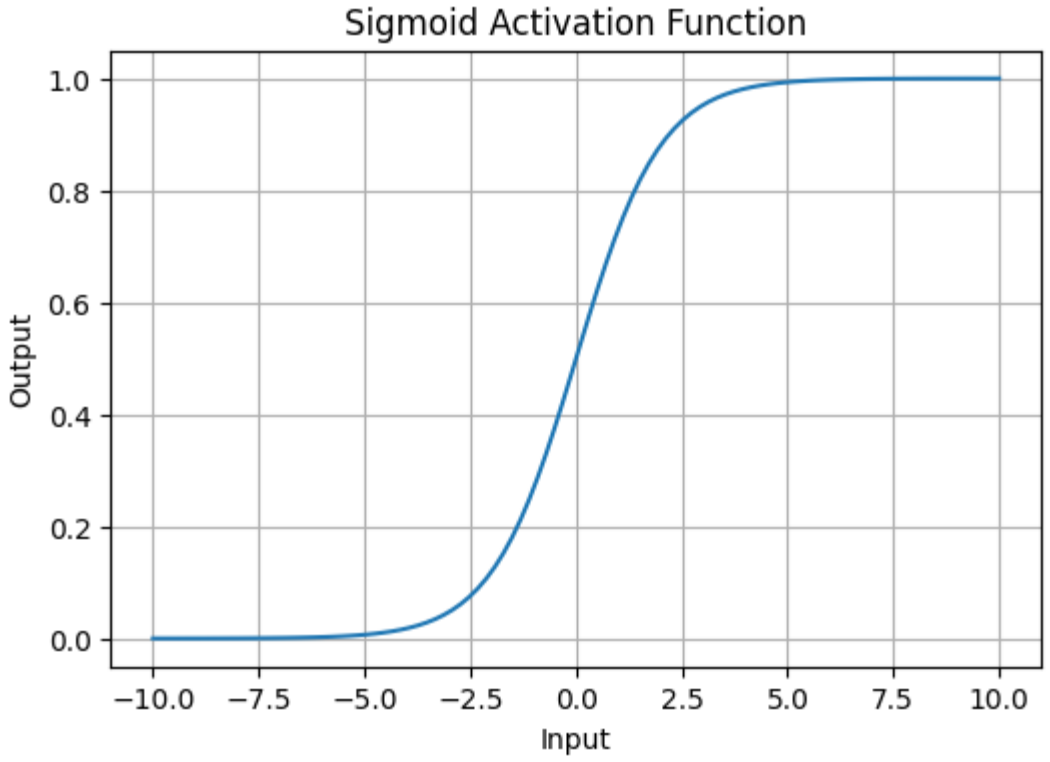
- A linear transform is basically the identity function,

$$f(x) = W * x$$

- Dependent variable has a direct, proportional relationship with the independent variable.
- It means the function passes the signal through unchanged.
- Used in the input layer of neural networks



ACTIVATION FUNCTIONS-SIGMOID

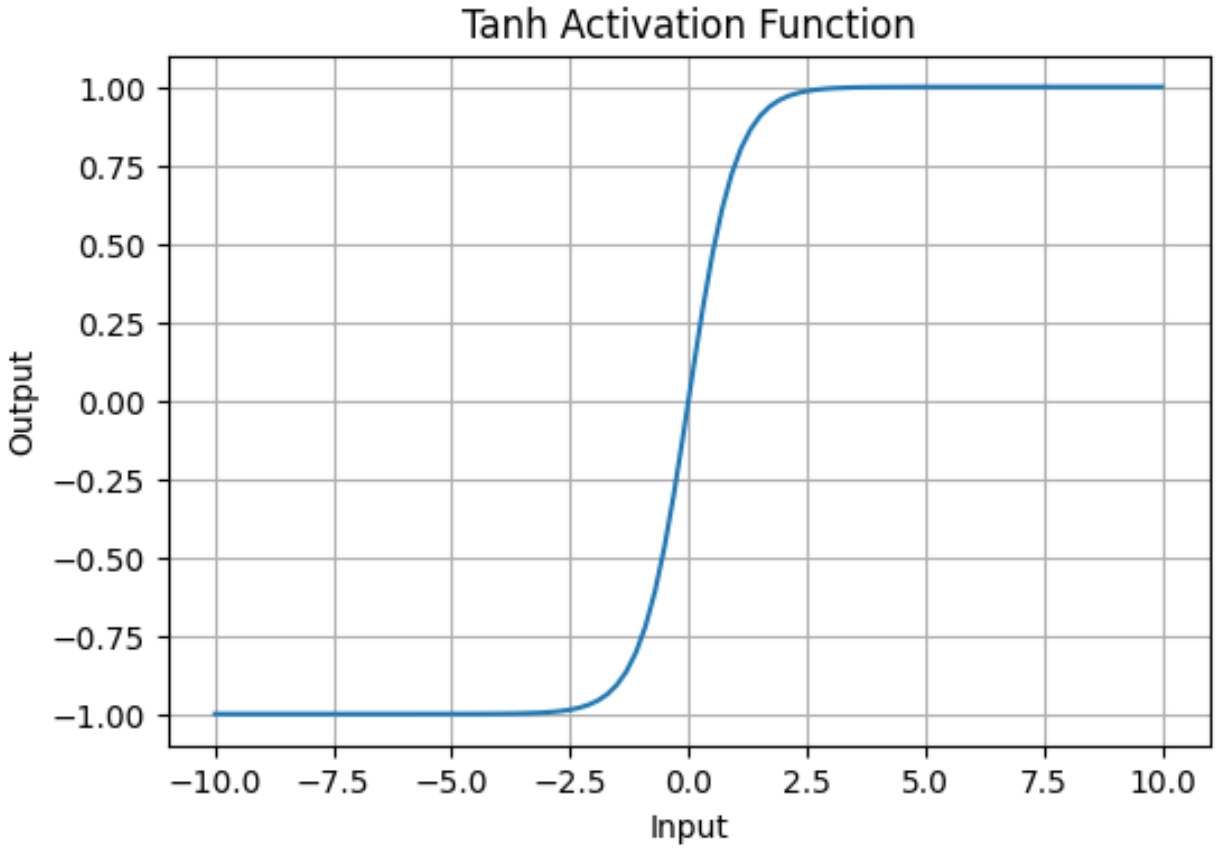


$$A = \frac{1}{1+e^{-x}}$$

- Sigmoids can reduce extreme values or outliers in data without removing them.
- A sigmoid activation function outputs an independent probability for each class.
- A sigmoid function is a machine that converts independent variables of near infinite range into simple probabilities between 0 and 1, and most of its output will be very close to 0 or 1.



ACTIVATION FUNCTIONS-TANH



- Hyperbolic tangent function
- Just as the tangent represents a ratio between the opposite and adjacent sides of a right triangle, tanh represents the ratio of the hyperbolic sine to the hyperbolic cosine:
- tanh(x) = sinh(x) / cosh(x).
- normalized range of tanh is –1 to 1.
- Advantage -it can deal more easily with negative numbers



ACTIVATION FUNCTIONS-TANH

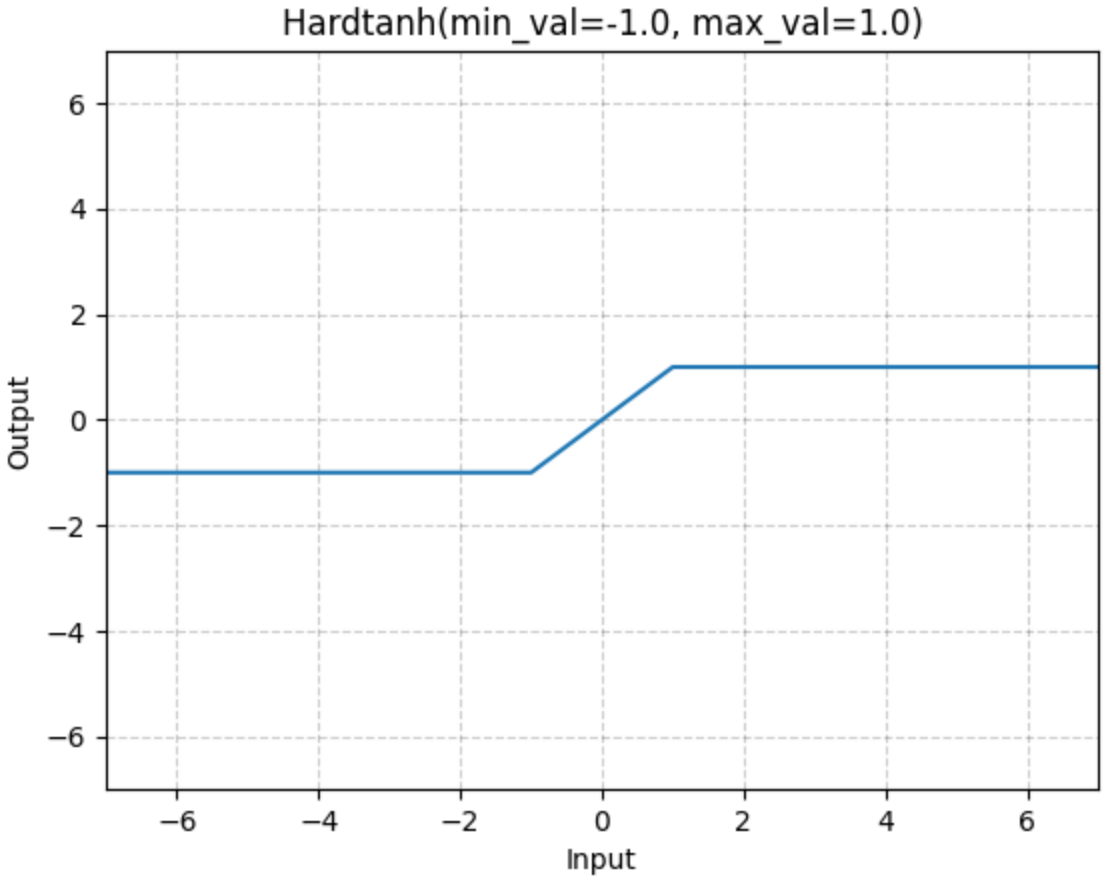
$$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1.$$

$$\tanh(x) = 2 \times \text{sigmoid}(2x) - 1$$

- **Non-linear**: Enables modeling of complex data patterns.
- **Use in Hidden Layers**: Commonly used in hidden layers due to its zero-centered output, facilitating easier learning for subsequent layers.



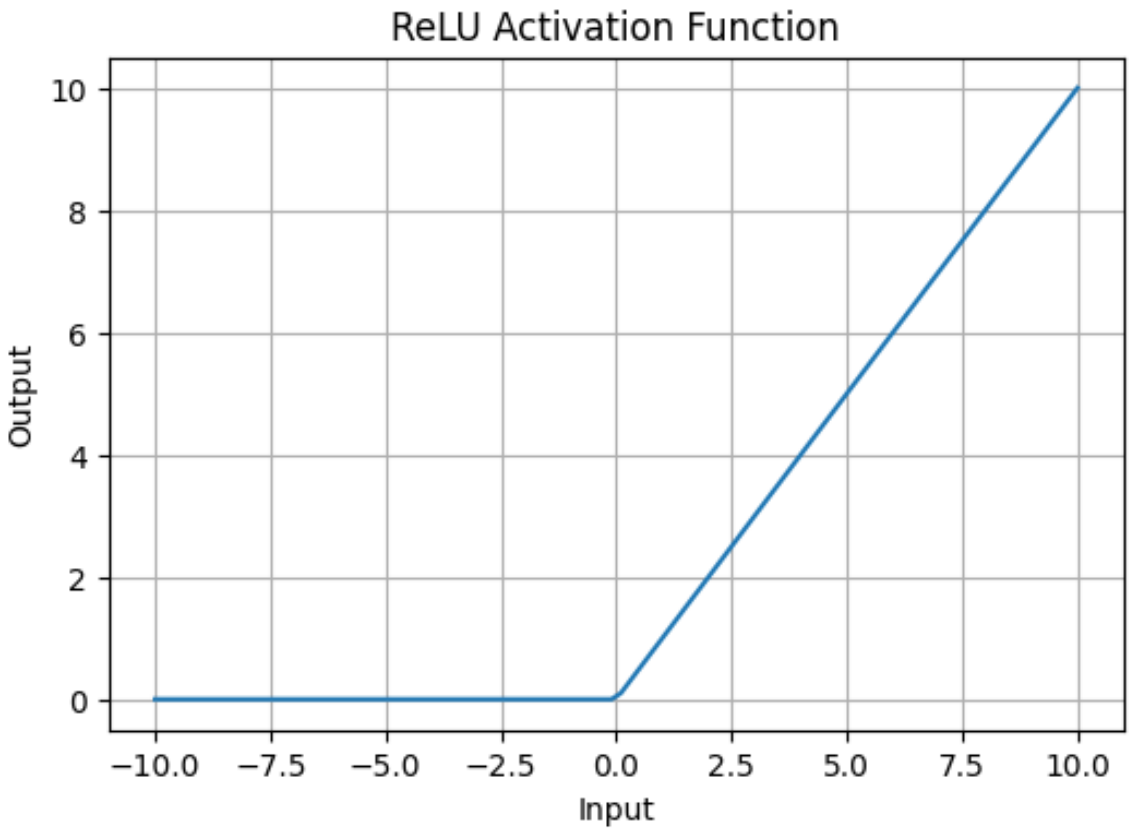
ACTIVATION FUNCTIONS-HARD TANH



Hard Tanh-

- Similar to tanh, hard tanh simply applies hard caps to the normalized range.
- Anything more than 1 is made into 1, and anything less than –1 is made into –1.
- This allows for a more robust activation function that allows for a limited decision boundary.

ACTIVATION FUNCTIONS-RECTIFIED LINEAR UNIT



- **ReLU-**
  - More interesting transform that activates a node only if the input is above a certain quantity.
  - While the input is below zero, the output is zero, but when the input rises above a certain threshold, it has a linear relationship with the dependent variable f(x) = max(0, x)



ACTIVATION FUNCTIONS-RECTIFIED LINEAR UNIT

- **ReLU-**
  - **Value Range**: $[0, \infty)$,meaning the function only outputs non-negative values.
  - **Nature**: It is a non-linear activation function, allowing neural networks to learn complex patterns and making backpropagation more efficient.
  - **Advantage over other Activation:** Less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.
  - Compared to the sigmoid and tanh activation functions, the ReLU activation function does not suffer from vanishing gradient issues



ACTIVATION FUNCTIONS-RECTIFIED LINEAR UNIT

The Dying ReLU problem—when inputs approach zero, or are negative, the gradient of the function becomes zero, the network cannot perform backpropagation and cannot learn

- **What causes the Dying ReLU problem?**
- **(i) High learning rate**
- Let us first look at the equation for the update step in backpropagation:

$$W_{ij}^{*} = W_{ij} - \alpha\left(\frac{\partial E}{\partial W_{ij}}\right)$$

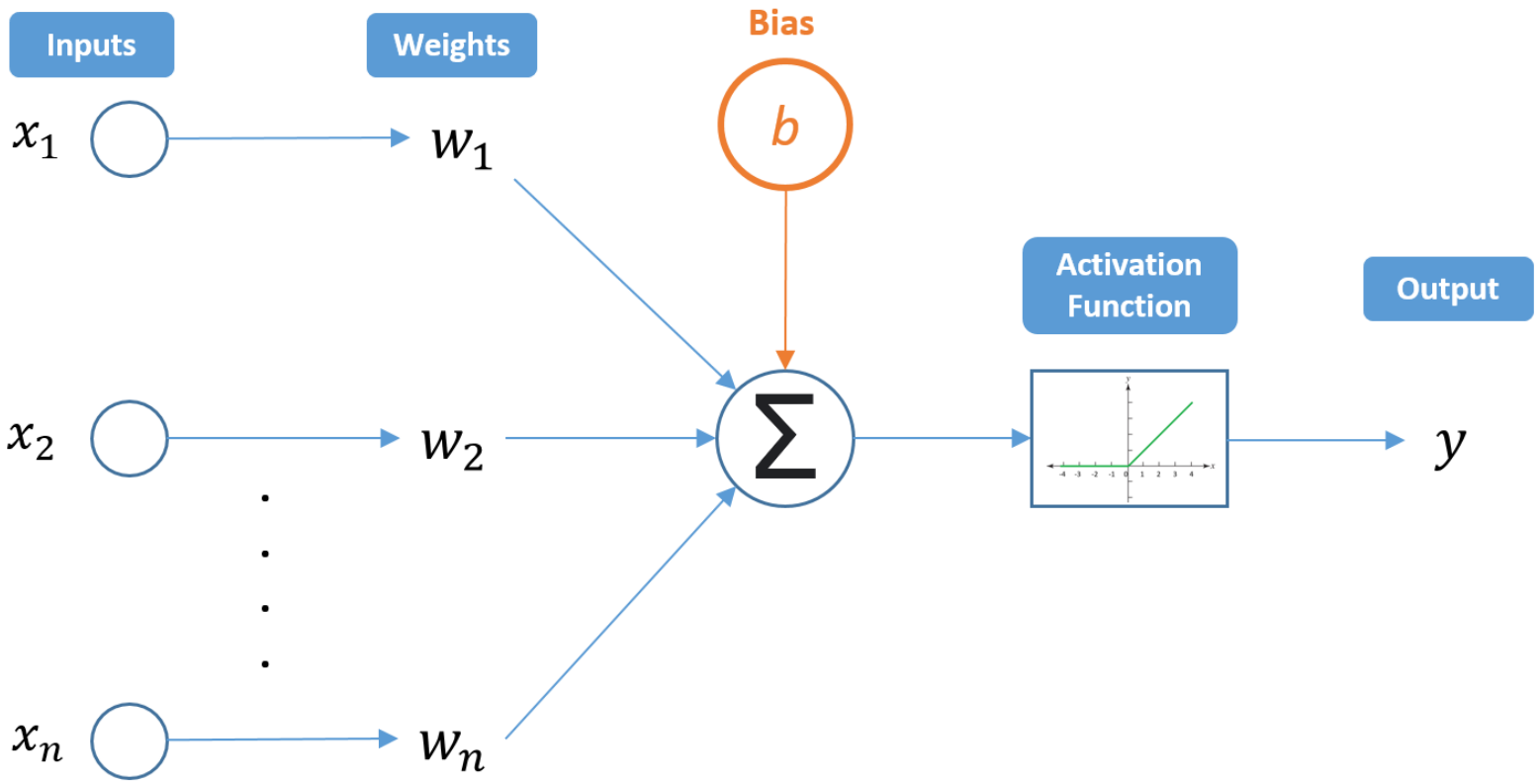
New Weight — Old Weight — Learning Rate — Derivative of Error with respect to Weight



ACTIVATION FUNCTIONS-RECTIFIED LINEAR UNIT

The Dying ReLU problem

- **(ii) Large negative bias**



ACTIVATION FUNCTIONS-RECTIFIED LINEAR UNIT

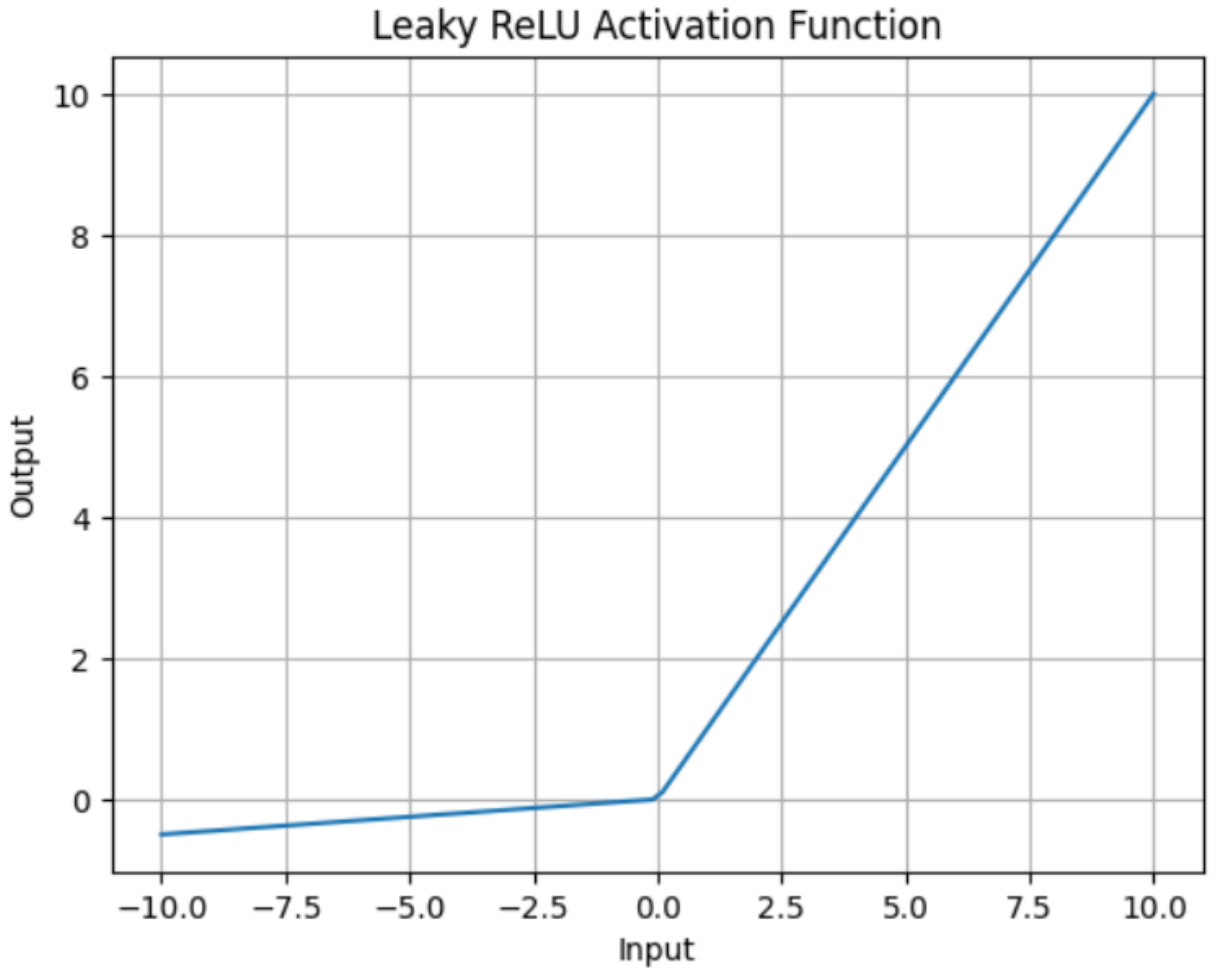
- **How to solve the Dying ReLU problem?**
  - (i) Use of a lower learning rate
  - (ii) Variations of ReLU



ACTIVATION FUNCTIONS-LEAKY RELU

- A strategy to mitigate the "dying ReLU"
- As opposed to having the function being zero when x < 0, the leaky ReLU will instead have a small negative slope (e.g., "around 0.01").
- Some success has been seen in practice with this ReLU variation, but results are not always consistent
- The equation is :

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

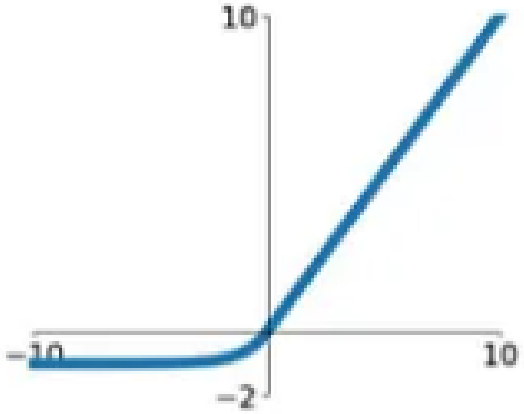


ACTIVATION FUNCTIONS-ELU

- **ELU(Exponential Linear Units):** ELU is also a variation of ReLU, with a better value for x<0.
- It also has the same properties as ReLU along with:
  (a.) No Dead ReLU Situation.
  (b.) Closer to Zero mean Outputs than Leaky ReLU.
  (c.) More Computation because of Exponential Function.

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



ACTIVATION FUNCTIONS- PARAMETRIC RELU (PRELU)

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

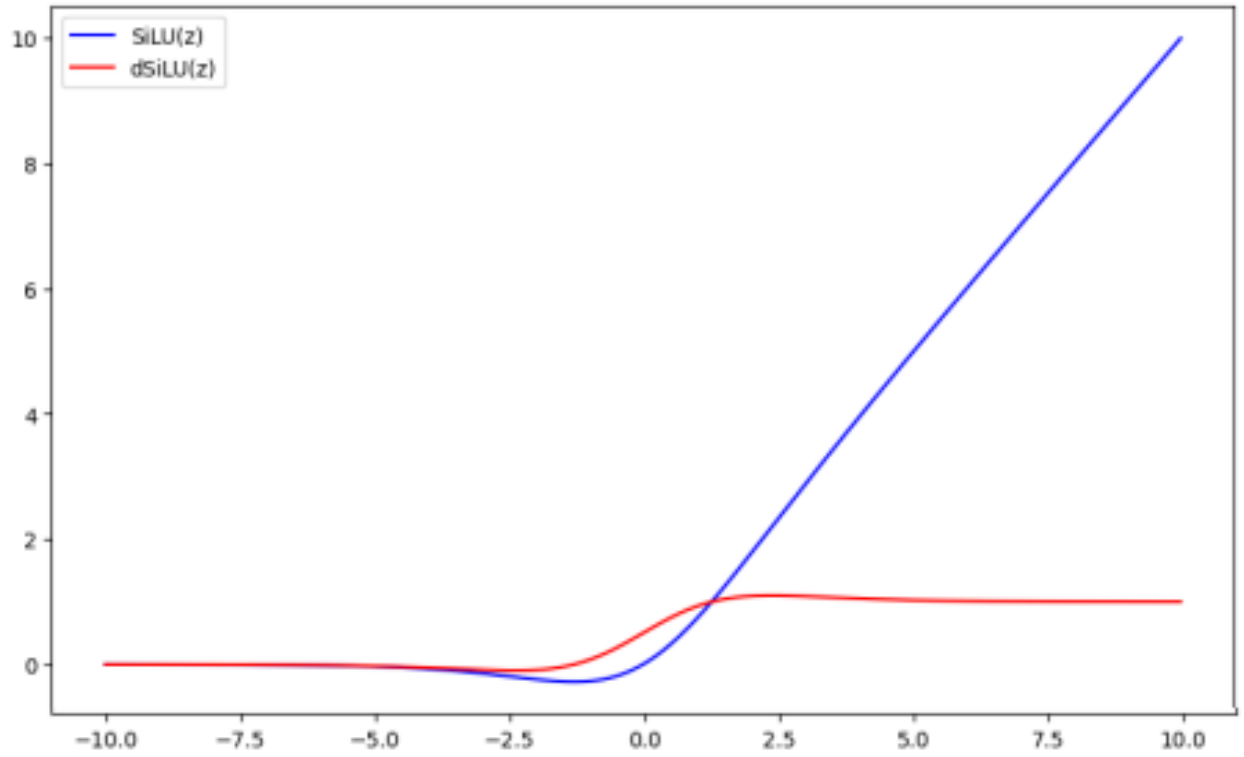
- α - trainable parameter for PReLU.



ACTIVATION FUNCTIONS- SIGMOID-WEIGHTED LINEAR UNIT (SILU)

- Proposed in 2017 for reinforcement learning[17], and its generalization, Swish
- combines both linear and sigmoid behaviors

$$\mathrm{SiLU}(z) = z \cdot \frac{1}{1+e^{-z}}$$





ACTIVATION FUNCTIONS- SWISH

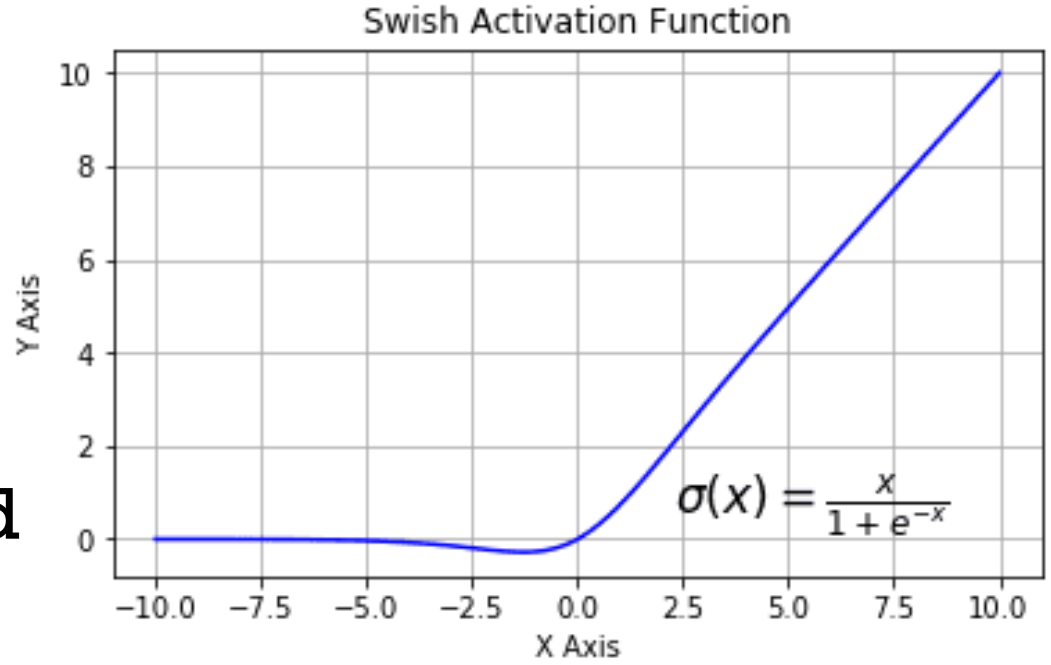
- The Swish function is a newer activation function.
- It stands out because it is non-monotonic, implying that the function's value could decrease even when the input values rise.
- Google developed Swish, and it occasionally performs better than the ReLU function.
- Swish has proven to be more effective than ReLU for deep architectures with more than 40 hidden layers.
- Recommended choice for very deep models



ACTIVATION FUNCTIONS- SWISH

- It is defined mathematically as-

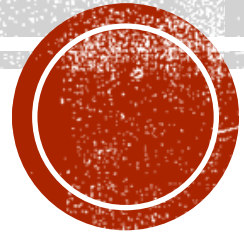
$$\text{Swish}(z) = z \cdot \frac{1}{1 + e^{-\beta z}}$$



- where $\beta \in [0,\infty[$ is either a user-defined constant or a trainable parameter.
- For $\beta = 0$, the Swish function reduces to the linear function f(z) = z, and for $\beta \to \infty$, it approaches the ReLU function.



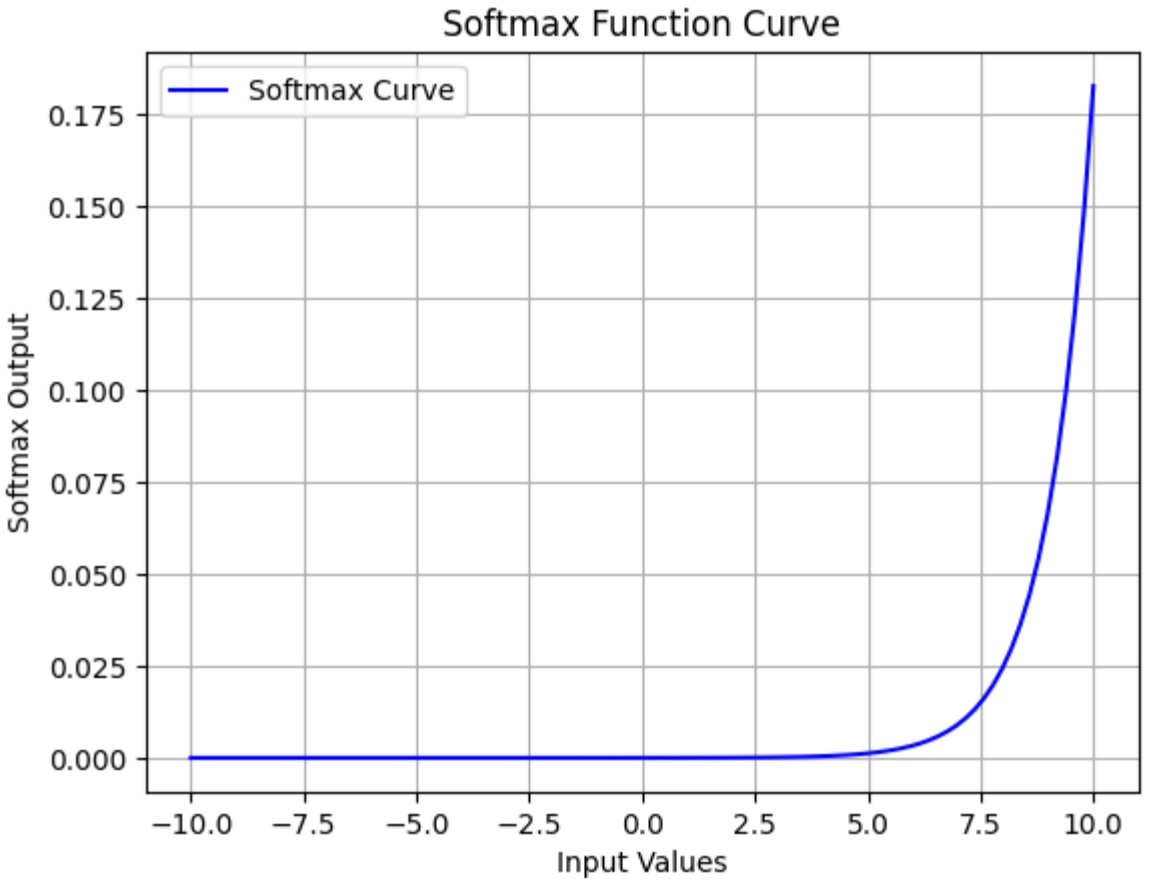
EXPONENTIAL LINEAR UNITS



ACTIVATION FUNCTIONS-SOFTMAX

Softmax-

- Also known as - softargmax or normalized exponential function
- Allows us to express our inputs as a discrete probability distribution
- a generalization of logistic regression in as much as it can be applied to continuous data (rather than classifying binary) and can contain multiple decision boundaries.
- Designed to handle multi-class classification problems



ACTIVATION FUNCTIONS-SOFTMAX

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

σ = softmax

$\vec{z}$ = input vector

e^{z_i} = standard exponential function for input vector

K = number of classes in the multi-class classifier

e^{z_j} = standard exponential function for output vector

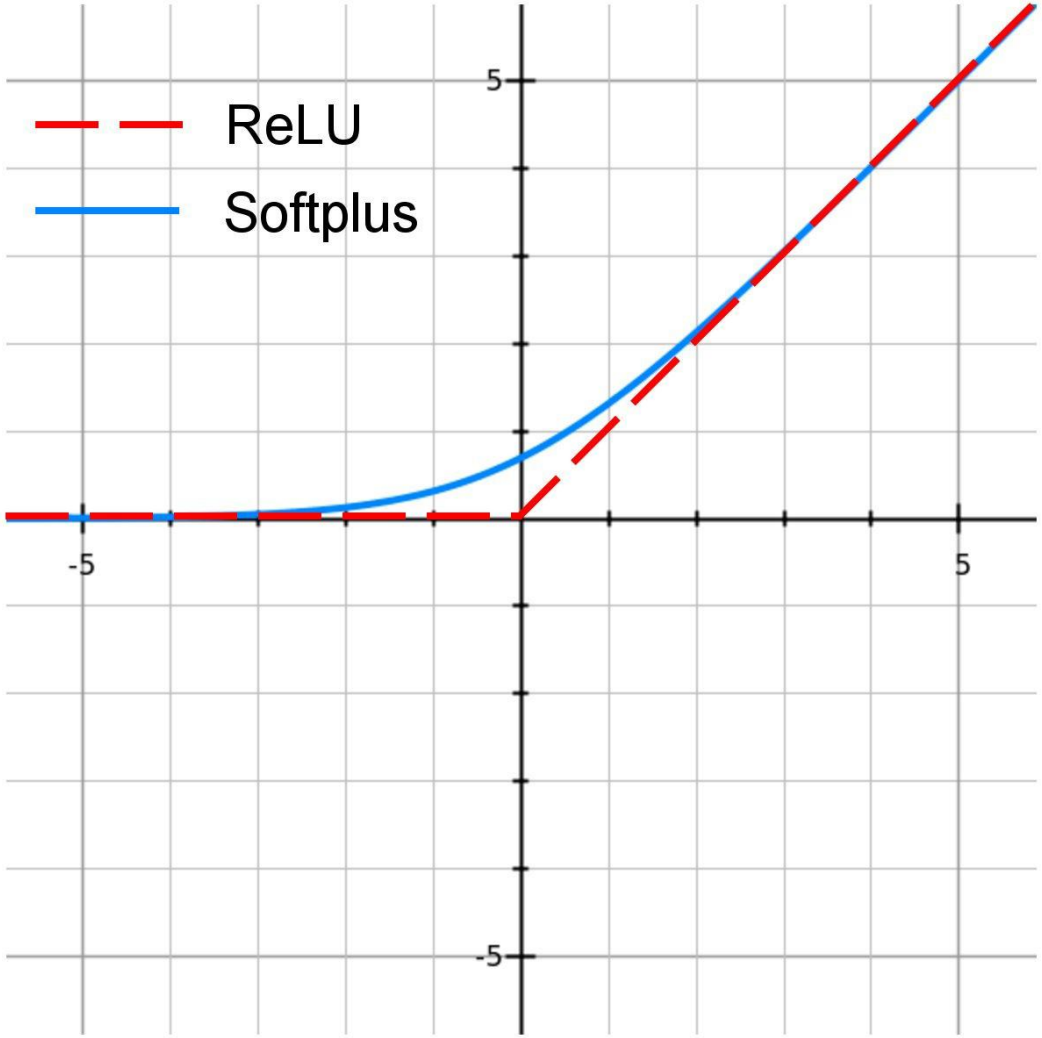
e^{z_j} = standard exponential function for output vector

- Generally used at the output layer of a classifier
- Defined as follows:
  - for each value in input vector, the Softmax value is the exponent of the individual input divided by a sum of the exponents of all the inputs.
- Returns the probability distribution over mutually exclusive output classes.



ACTIVATION FUNCTIONS-SOFTPLUS

- This activation function is the "smooth version of the ReLU,"
- Compare this plot to the ReLU - shows that the softplus activation function
- $(f(x) = \ln[\ 1 + \exp(x)\ ])$
- Has a similar shape to the ReLU.



ACTIVATION FUNCTIONS-SOFTPLUS

- **Range**: The function outputs values in the range $(0, \infty)$, similar to ReLU, but without the hard zero threshold that ReLU has.
- **Smoothness**: Softplus is a smooth, continuous function, meaning it avoids the sharp discontinuities of ReLU which can sometimes lead to problems during optimization.

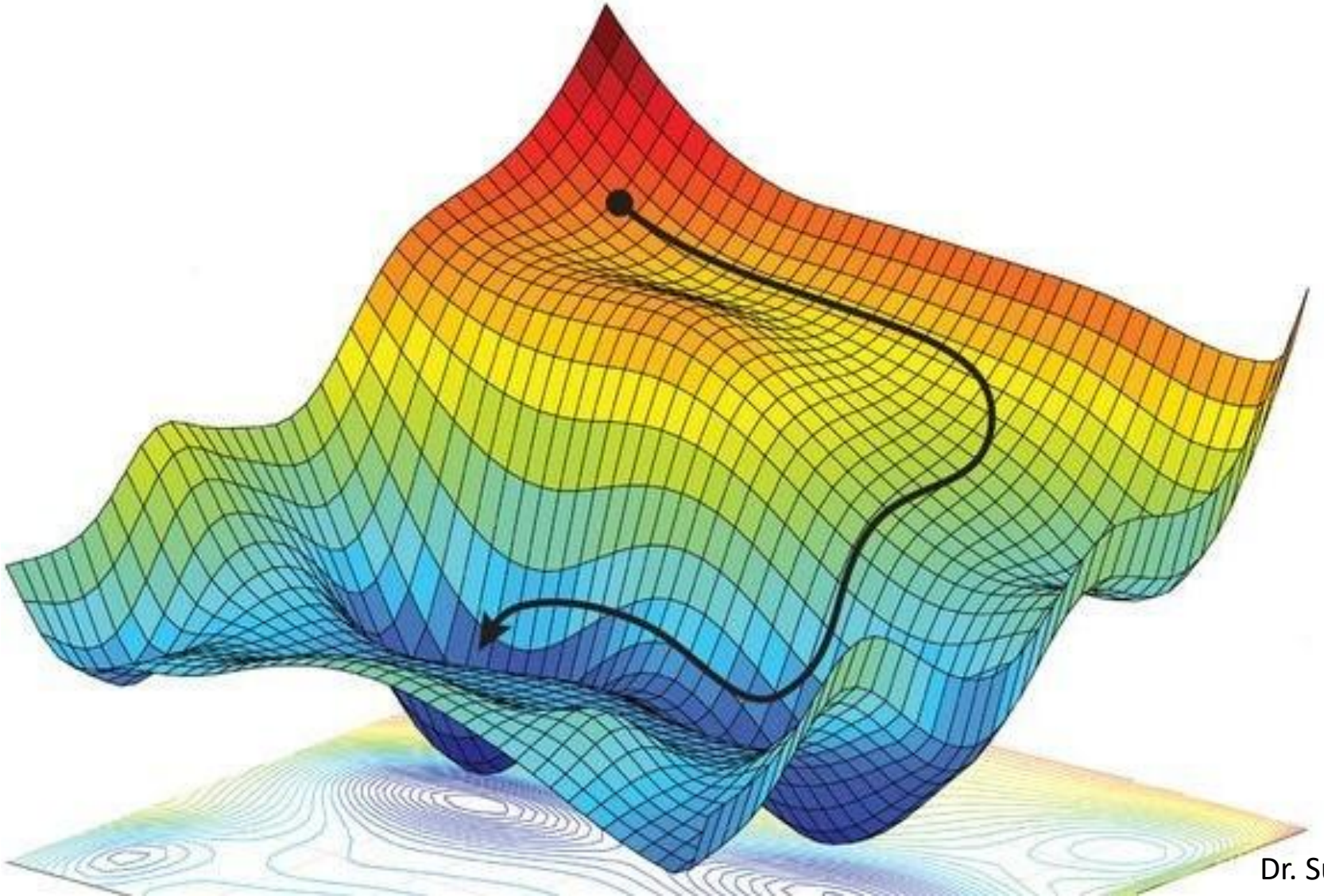


ACTIVATION FUNCTIONS- CASE STUDY

- **Binary Classification Case-**
  - Sigmoid the most suitable activation function in output layer
  - For the hidden layers, the sigmoid should be avoided and replaced by ReLU or one of its variants
- **Multi-class Classification Case-**
  - Most appropriate activation function for output units is the softmax
- **Case of Regression Problems-**
  - The recommended activation functions are ReLU (or one of its variants or alternatives) for the hidden layers, and a linear activation function for the output layer.



Loss functions



Loss functions

How good is your neural network?

Loss functions

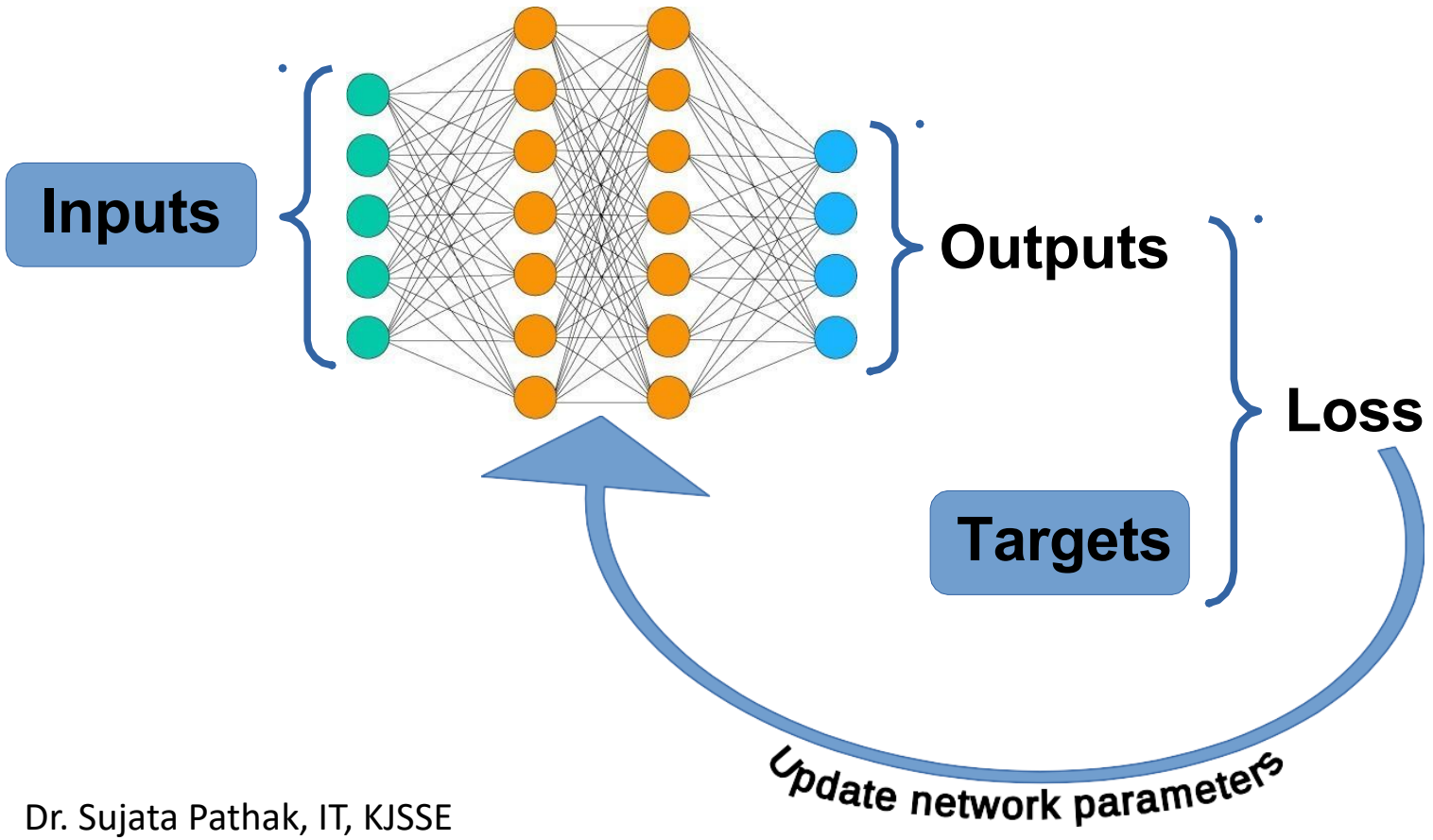
How good is your neural network?

Inputs

e.g.
- *Image (pixel values)*
- *Sentence (encoded)*
- *State (for RL)*

Targets

e.g.
- *'Cat'/'Dog' (encoded)*
- *Next word (encoded)*
- *TD(0) (for RL)*



LOSS FUNCTIONS

- ***Loss function*** *and* ***Cost function*** *are often used interchangeably but they are different:*
- *The* ***loss function*** *measures the network performance* ***on a single datapoint***
- *The* ***cost function*** *is the* ***average of the losses*** *over the entire training dataset*
- *Our goal is to* ***minimize the cost function****.*
- *In reality, we actually use* ***batch losses*** *as a proxy for the cost function.*

LOSS/COST FUNCTIONS

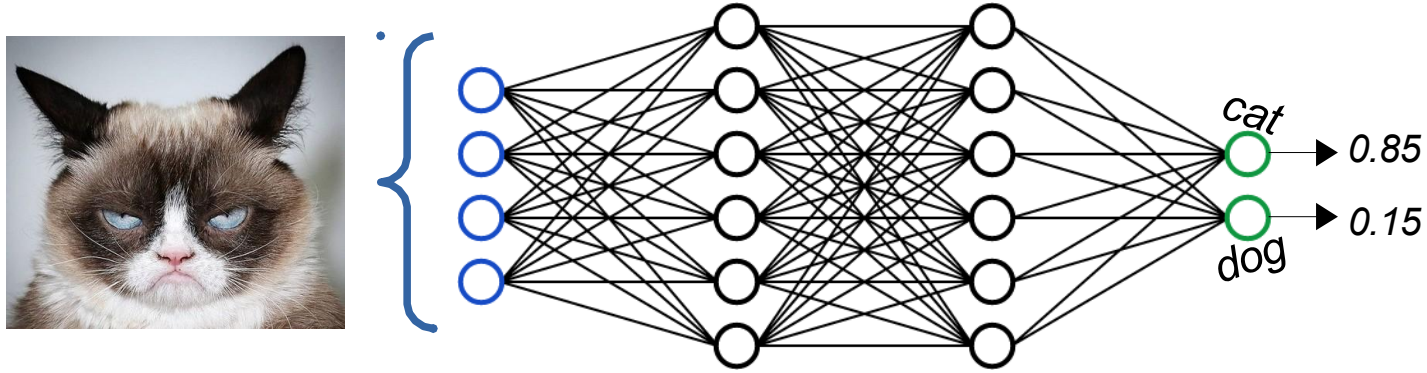
Stuff to remember

- The cost function distils the performance of the network down to a **single scalar number**. Generally (although not necessarily) **loss ≥ 0** (so **cost ≥ 0** also)
- cost=0 ⇒ perfect performance (on the given dataset)
- The **lower the value of the cost function, the better the performance** of the network.
- Hence gradient descent
- A cost function must **faithfully represent the purpose** of the network.

Loss/Cost functions

Some stuff to remember

The cost function distils the performance of the network down to a ***single scalar number****.*
Generally (although not necessarily) ***loss*** $\geq$ ***0*** *(so* ***cost*** $\geq$ ***0*** *also)*
cost=0 $\Rightarrow$ *perfect performance (on the given dataset)*
The ***lower the value of the cost function, the better the performance*** *of the network.*
Hence gradient descent
A cost function must ***faithfully represent the purpose*** *of the network.*



Accuracy*: 1*
Loss (CCE)*: 0.07*

Loss/Cost functions

Loss functions

Classification

Maximum likelihood

Binary cross-entropy (aka log loss)

Categorical cross-entropy

Regression *(i.e. function approximation)*

Mean Squared Error Mean Absolute Error

Huber Loss

Classification loss functions

Loss functions

Classification

- *Maximum likelihood*
- *Binary cross-entropy (aka log loss)*
- *Categorical cross-entropy*

Loss Functions

Loss Function Notation-

"N" - number of samples (set of inputs with corresponding outcomes) that have been gathered.
"P" - number of input features gathered
"M" - the number of output features that have been observed.
(X,Y) to denote the input and output data collected
N such pairs where the input is a collection of P values and the output Y is a collection of M values.
Denote the ith pair in the dataset as Xi and Yi .

We will use $\hat{Y}$ to denote the output of the neural net.
$\hat{Y}$ - network's guess at Y and therefore it will also have M features.

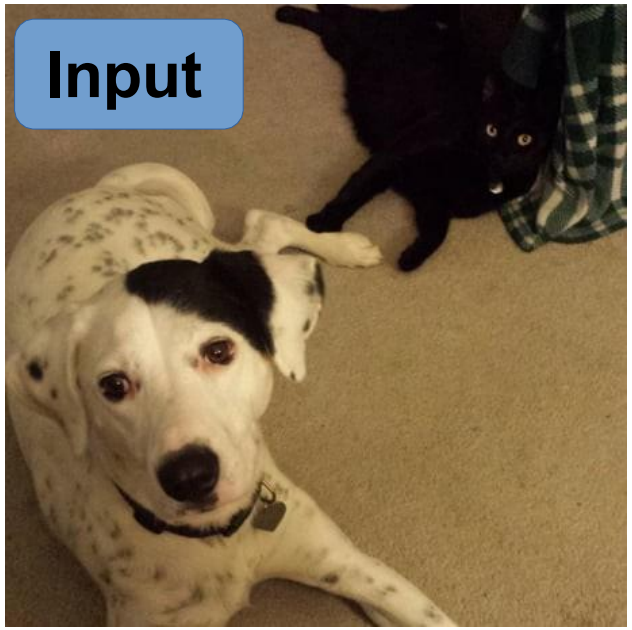
Loss Functions

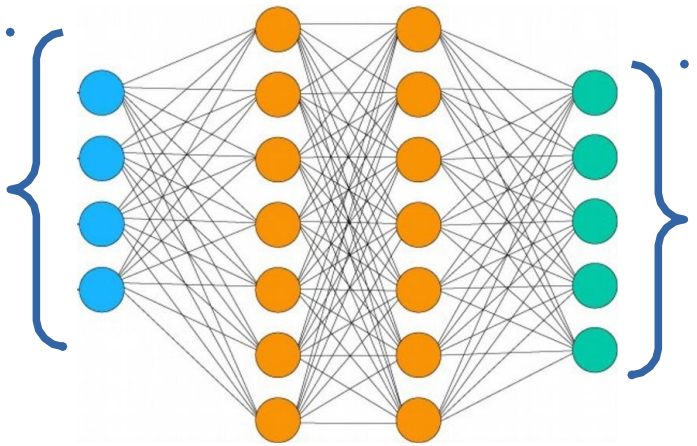
- Use the notation $h(X_i) = \hat{Y}_i$ to denote the neural network transforming the input to give the output .
- When referring to *j th* output feature, use it as a subscript firmly linking the notation to a matrix where the rows are different data points and the columns are the different unique features. Thus $y_{i,j}$ refers to the jth feature observed in the ith sample collected.
- *R*epresent the loss function by *L(W,b)*.

$$h_{w,b}(X) = \hat{Y}$$

Classification loss functions

Maximum likelihood





Cat - .5
Dog - .9
Bear - .2
Hitler - .6
Putin - .1

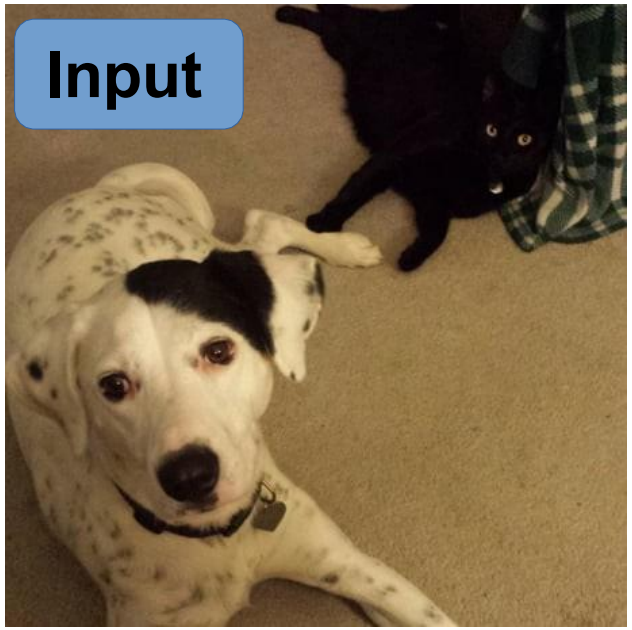
Target

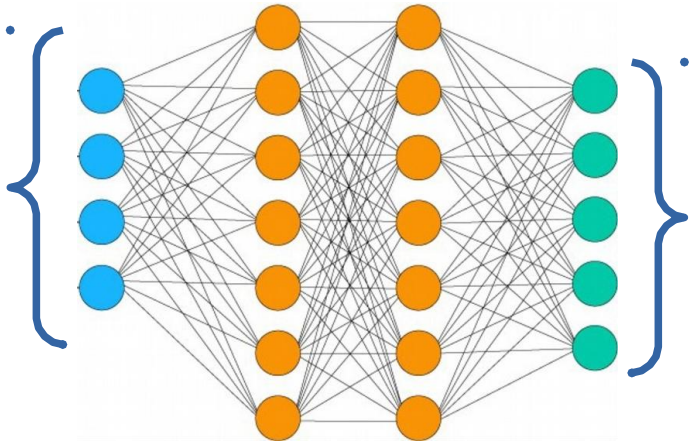
(1, 1, 0, 0, 0)

Cat -
Dog -
Bear -
Hitler -
Putin -

Classification loss functions

Maximum likelihood





- Cat - Dog - Bear - Hitler - Putin
(.5, .9, .2, .6, .1)

$$Loss = -\frac{1}{N}\sum_{i=1}^{N} y_i \cdot (p(y_i)) + (1 - y_i) \cdot (1 - p(y_i))$$

Target

(1, 1, 0, 0, 0)
Cat - Dog - Bear - Hitler - Putin -

$$= -1/5\ [\ (1*0.5 + 0*0.5) + (1*0.9 + 0*0.1) + (0*0.2 + 1*0.8) + (0*0.6 + 1*0.4) + (0*0.1 + 1*0.9)\ ]$$

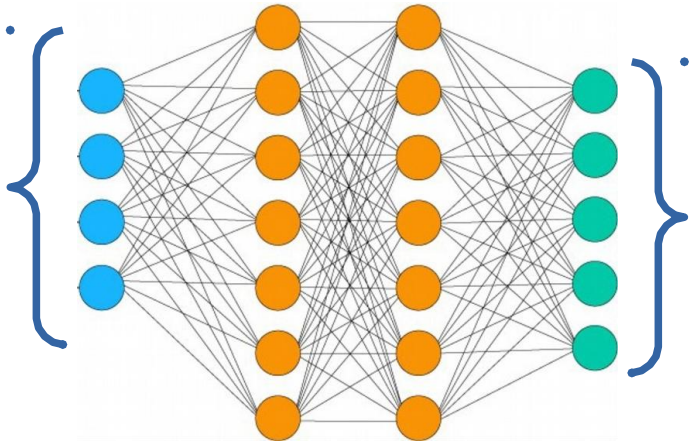
$$= -0.7$$

Classification loss functions

Binary cross-entropy (aka Log loss)

Torch.nn.BCELoss





- Cat - Dog - Bear - Hitler - Putin
(.2, .4, .8, .3, .9)

$$Loss = -\frac{1}{N}\sum_{i=1}^{N} y_i \cdot log(p(y_i)) + (1 - y_i) \cdot log(1 - p(y_i))$$

Target

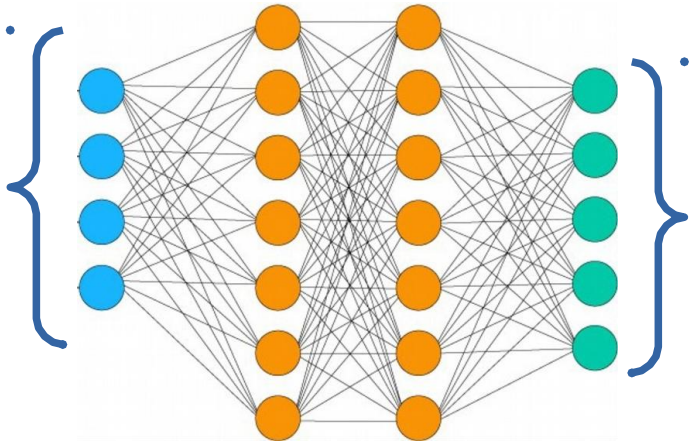
(0, 0, 1, 0, 1)
Cat - Dog - Bear - Hitler - Putin -

Classification loss functions

Binary cross-entropy (aka Log loss)

Torch.nn.BCELoss





- Cat - Dog - Bear - Hitler - Putin
(.2, .4, .8, .3, .9)

$$Loss = -\frac{1}{N}\sum_{i=1}^{N} y_i \cdot log(p(y_i)) + (1 - y_i) \cdot log(1 - p(y_i))$$

Target

(0, 0, 1, 0, 1)
Cat - Dog - Bear - Hitler - Putin

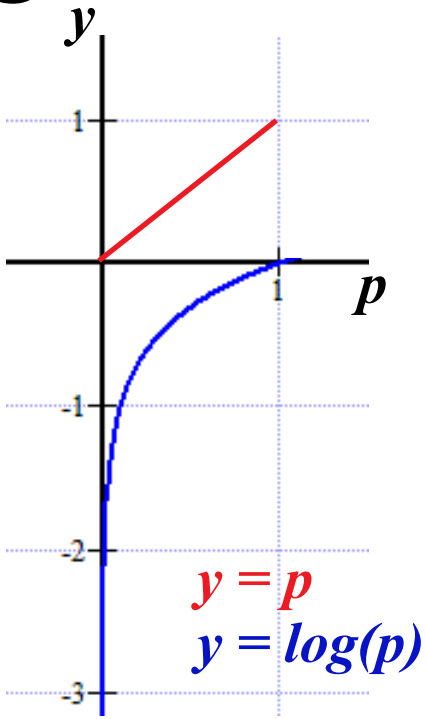
$$= -1/5\ [\ (1*log(0.8)) + (1*log(0.6)) + (1*log(0.8)) + (1*log(0.7)) + (1*log(0.9))\ ]$$

$$= 0.28$$

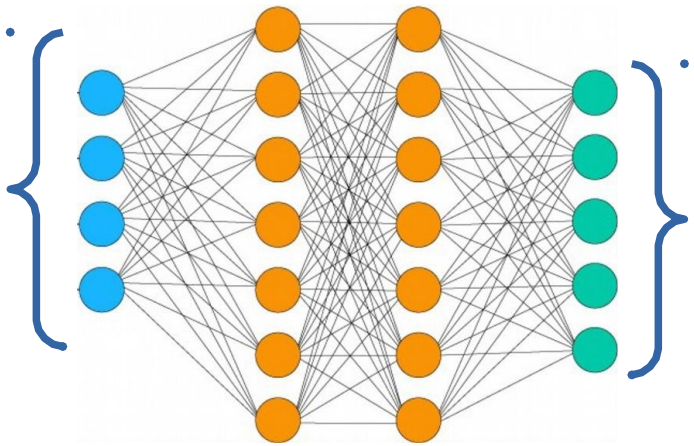
Classification loss functions

Binary cross-entropy (aka Log loss)

Torch.nn.BCELoss







- Cat - Dog - Bear - Hitler - Putin
(.2, .4, .8, .3, .9)

$$Loss = -\frac{1}{N}\sum_{i=1}^{N} y_i \cdot log(p(y_i)) + (1 - y_i) \cdot log(1 - p(y_i))$$

Target

(0, 0, 1, 0, 1)
Cat - Dog - Bear - Hitler - Putin -

$$= -1/5\ [\ (1*log(0.8)) + (1*log(0.6)) + (1*log(0.8)) + (1*log(0.7)) + (1*log(0.9))\ ]$$

$$= 0.28$$

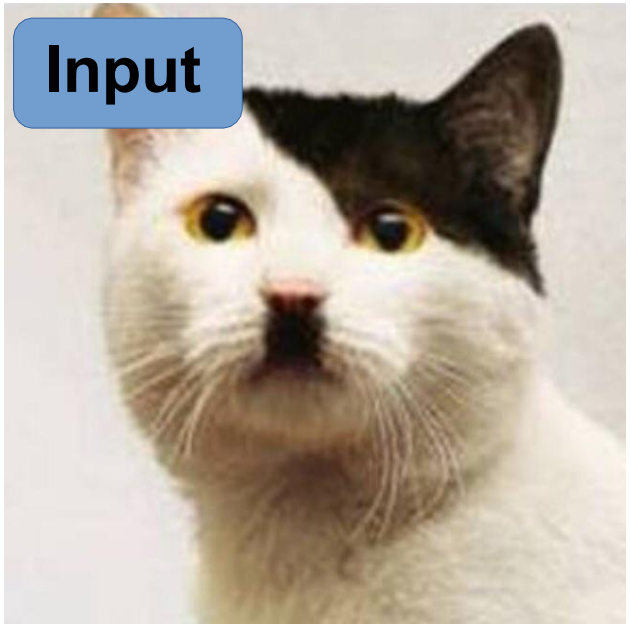
Classification loss functions

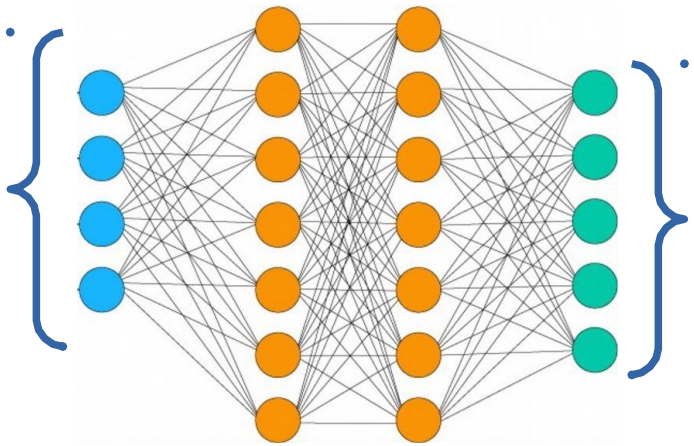
Categorical cross-entropy

Torch.nn.NLLLoss / Torch.nn.CrossEntropyLoss

This one does the softmax for you

$$= \log\left(\frac{\exp(x_i)}{\sum_j \exp(x_j)}\right)$$





(.8, 0, 0, .2, 0)

Cat - Dog - Bear - Hitler - Putin

Must be a probability distribution

$$Loss = -\sum_{i=1}^{N} y_i \cdot log(p(y_i))$$

Target

(1, 0, 0, 0, 0)

Cat - Dog - Bear - Hitler - Putin

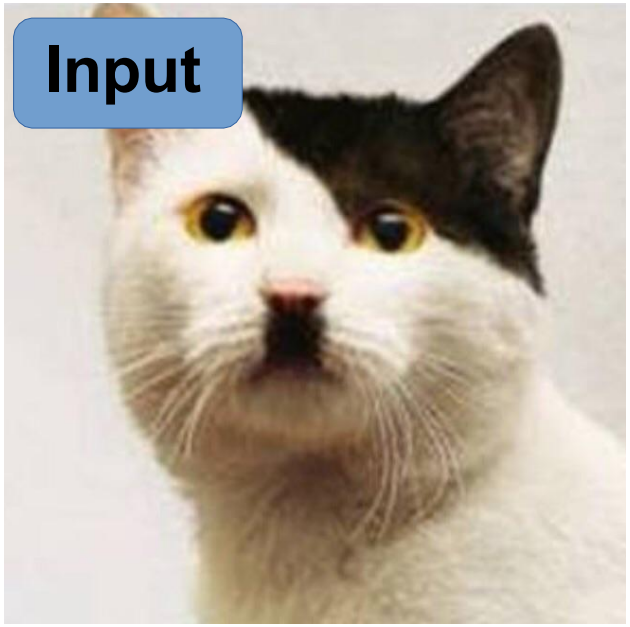
$$= -[\ 1*log(0.8)\ ]$$

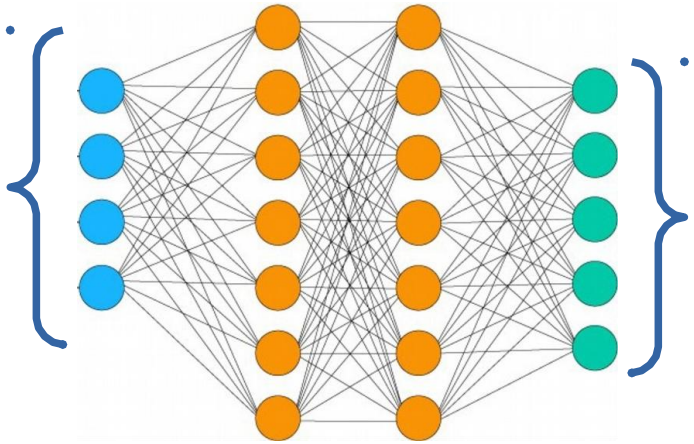
$$= 0.22$$

Classification loss functions

Categorical cross-entropy

Torch.nn.NLLLoss / Torch.nn.CrossEntropyLoss





(.8, 0, 0, .2, 0)

- Cat - Dog - Bear - Hitler - Putin

This one does the softmax for you

$$= \log\left(\frac{\exp(x_i)}{\sum_j \exp(x_j)}\right)$$

Must be a probability distribution

$$Loss = -\sum_{i=1}^{N} y_i \cdot log(p(y_i))$$

Target

(1, 0, 0, 0, 0)

Cat - Dog - Bear - Hitler - Putin

$$= -[\,1 * log(0.8)\,]$$

$$= 0.22$$

This loss function can also be used when you have a single binary (e.g. yes/no) output

Loss Functions

- *Loss Functions for Classification-*
  - Hinge loss- most commonly used loss function when the network must be optimized for a hard classification
  - Also seen in a class of models called maximum-margin classification models (e.g., support vector machines)
  - equation for hinge loss when data points must be categorized as –1 or 1

$$L(W, b) = \frac{1}{N}\sum_{i=1}^{N} max(0, 1 - y_{ij} \times \hat{y}_{ij})$$

  - Mostly used for binary classifications

Loss Functions

Loss Functions for Classification-

- Logistic loss- used when probabilities are of greater interest than hard classifications.

$$P(y_i = 1 \mid X_i; \mathbf{W}, \mathbf{b}) = h_{\mathbf{W},\mathbf{b}}(X_i)$$

$$P(y_i = 0 \mid X_i; \mathbf{W}, \mathbf{b}) = 1 - h_{\mathbf{W},\mathbf{b}}(X_i)$$

$$P(y_i \mid X_i; \mathbf{W}, \mathbf{b}) = (h_{\mathbf{W},\mathbf{b}}(X_i))^{y_i} \times (1 - h_{\mathbf{W},\mathbf{b}}(X_i))^{1-y_i}$$

$$L(W, b) = \prod_{i=1}^{N} \hat{y}_i^{y_i} \times (1 - \hat{y}_i)^{1-y_i}$$

Loss Functions

Loss Functions for Classification-

Negative log likelihood

$$L(W, b) = -\sum_{i=1}^{N} y_i \times \log \hat{y}_i + (1 - y_i) \times \log (1 - \hat{y}_i)$$

Extending the loss function from two classes to M classes gives us the following equation for logistic loss

$$L(W, b) = -\sum_{i=1}^{N} \sum_{j=1}^{M} y_{i,j} \times \log \hat{y}_{i,j}$$

Regression loss functions

Loss functions

Regression *(i.e. function approximation)*

- *Mean Squared Error*
- *Mean Absolute Error*
- *Huber Loss*

Loss Functions

Loss Functions for Regression-

Mean squared error loss

$$L(W, b) = \frac{1}{N}\sum_{i=1}^{N}\left(\hat{Y}_i - Y_i\right)^2$$

$$L(W, b) = \frac{1}{2N}\sum_{i=1}^{N}\sum_{j=1}^{M}\left(\hat{y}_{ij} - y_{ij}\right)^2$$

$$L(W, b) = \frac{1}{2N}\sum_{i=1}^{N}\sum_{j=1}^{M}\left(\hat{y}_{ij} - y_{ij}\right)^2$$

Loss Functions

Loss Functions for Regression-

Mean absolute error loss

$$L(W, b) = \frac{1}{2N} \sum_{i=1}^{N} \sum_{j=1}^{M} | \hat{y}_{ij} - y_{ij} |$$

Mean squared log error loss

$$L(W, b) = \frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{M} (\log \hat{y}_{ij} - \log y_{ij})^2$$

Mean absolute percentage error loss

$$L(W, b) = \frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{M} \frac{100 \times | \hat{y}_{ij} - y_{ij} |}{y_{ij}}$$

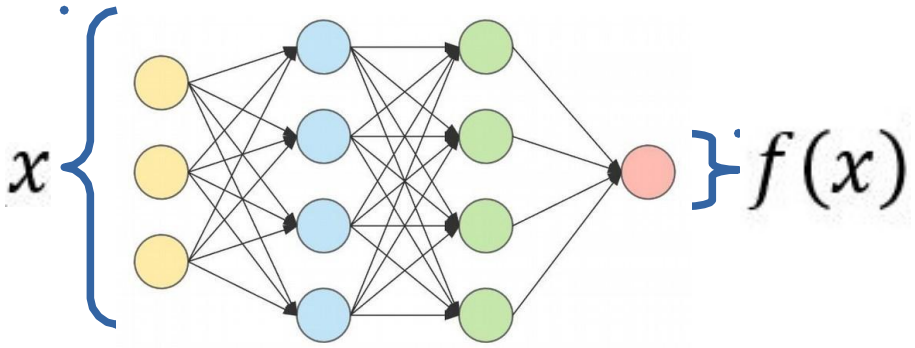
Regression loss functions

aka L2 loss

Mean squared error

Torch.nn.MSELoss

Input x



Target y

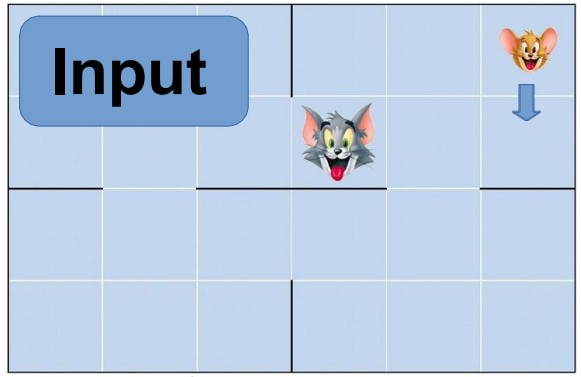
$$Loss = (y - f(x))^2$$

Regression loss functions

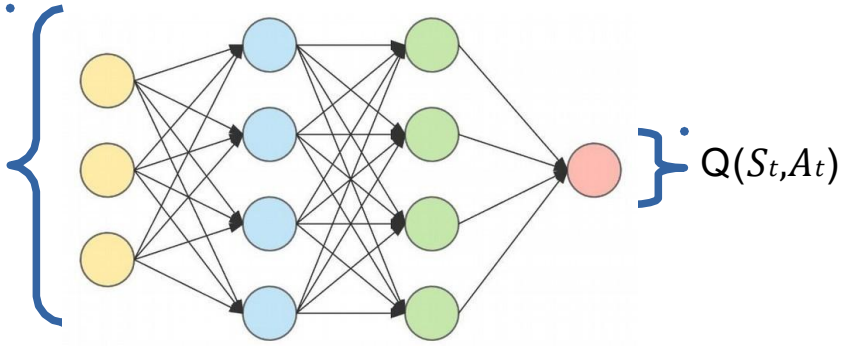
aka L2 loss

Mean squared error

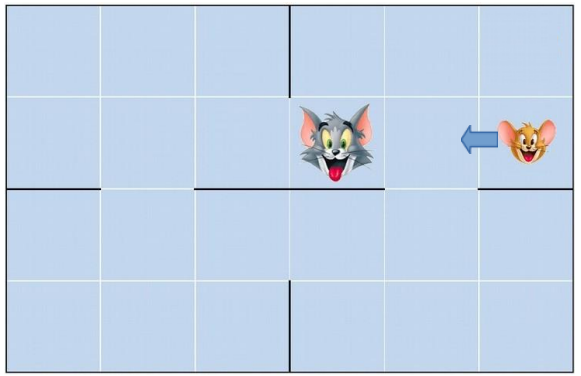
Torch.nn.MSELoss



state S_t & action A_t

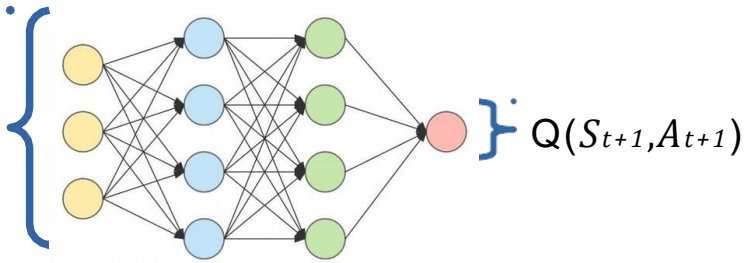


reward R_{t+1}



state S_{t+1} & [best] action A_{t+1}

Target $R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$



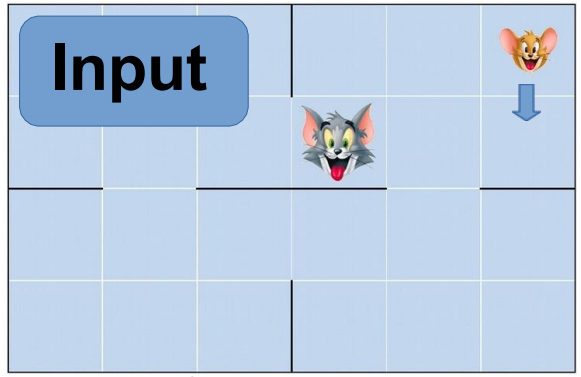
Regression loss functions

aka L2 loss

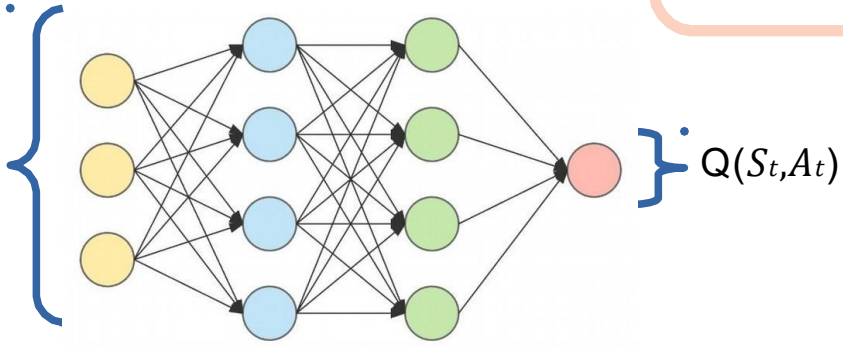
Mean squared error

Torch.nn.MSELoss

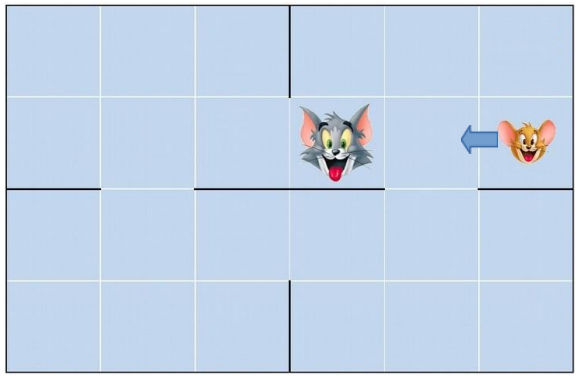
$$Loss = (y - f(x))^2$$
$$= \left((R_{t+1} + \gamma \max_a Q(S_{t+1}, a)) - Q(S_t, A_t) \right)^2$$



state S_t & action A_t

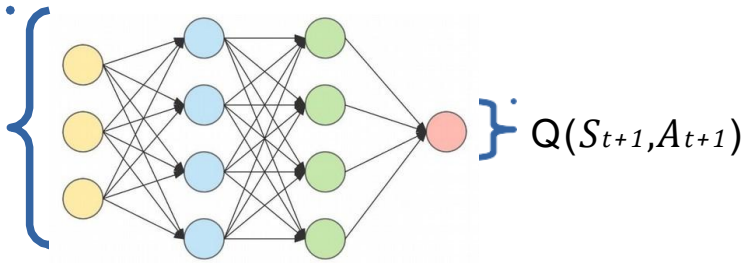


reward R_{t+1}



state S_{t+1} & [best] action A_{t+1}

Target $R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$



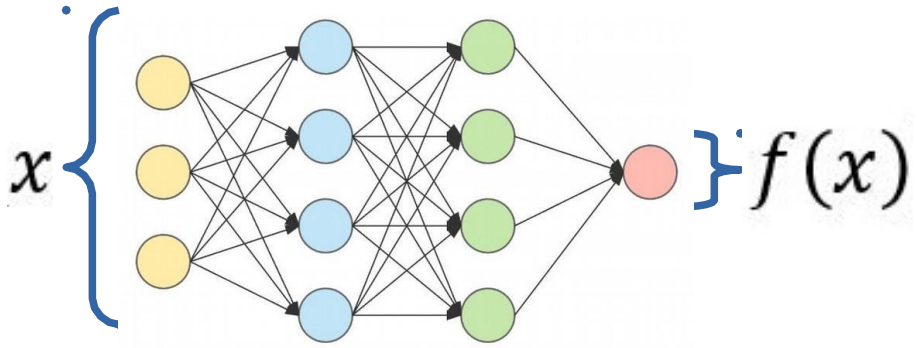
Regression loss functions

aka L1 loss

Absolute error

Torch.nn.L1Loss

Input x



Target y

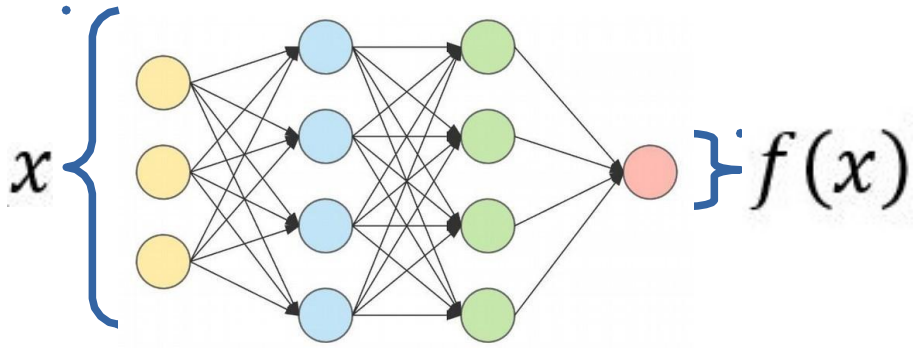
$$Loss = |y - f(x)|$$

Regression loss functions

Huber loss

Torch.nn.SmoothL1Loss

Input x

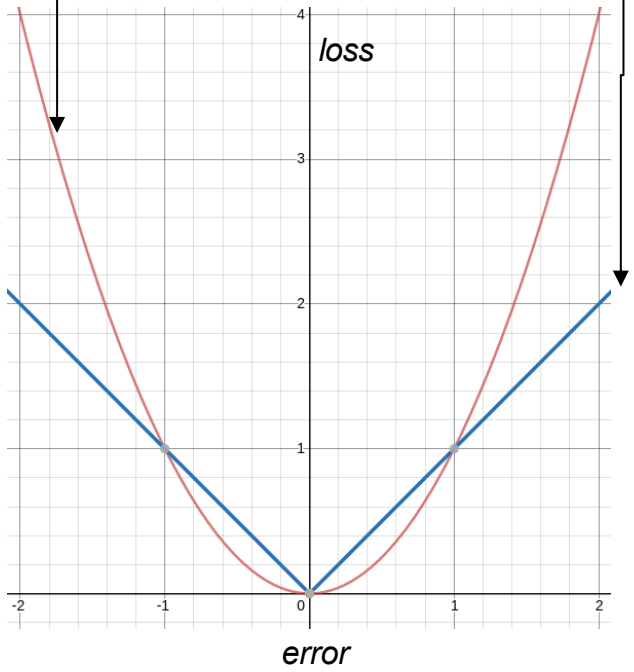


Target y

$$Loss = \begin{cases} \frac{1}{2}(y - f(x))^2, & if\ |y - f(x)| \leq \delta \\ \delta|y - f(x)| - \frac{1}{2}\delta^2, & otherwise \end{cases}$$

Huber loss combines the best features of MSE and AE:
- like MSE for small errors.
- like AE otherwise.

This avoids over-shooting:
- MSE has a **very steep gradient when the error is large** (i.e. for outliers) because of its quadratic form.
- AE has a **steep gradient when the error is small** because it doesn't have a 'flat' bottom.



LOSS FUNCTIONS

- Loss Functions for Reconstruction-
- equation for KL divergence

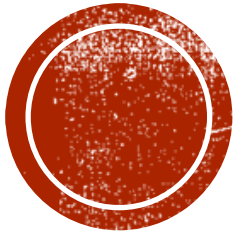
$$D_{KL}(Y \mid\mid \hat{Y}) = -\sum_{i=1}^{N} Y_i \times \log\left(\frac{Y_i}{\hat{Y}_i}\right)$$



Advantages and Disadvantages of Common Loss Functions

Loss Function	Advantages	Disadvantages
Cross-entropy loss	Effective in multi-class classification; Provides probabilistic interpretations; Easy to implement.	Sensitive to class imbalance; Can lead to overconfidence in predictions.
Binary Cross-Entropy Loss	Simple and efficient for binary classification; Provides clear probabilistic outputs.	Assumes outputs are independent; may not perform well with correlated data.
Weighted Cross-Entropy Loss	Helps mitigate class imbalance; Adjusts the importance of classes.	Requires careful tuning of weights; Increased computational overhead.
P-w Softmax with Cross-Entropy Loss	Effective for segmentation tasks; Maintains spatial context.	High computational and memory requirements; Complex gradients.
L1 Loss	Robust to outliers; Minimizes extreme values' impact.	Can lead to less smooth convergence during optimization.
Dice Loss	Well-suited for imbalanced datasets; Focuses on overlap rather than pixel-wise accuracy.	Gradients can be complex near boundaries and harder to optimize.
Focal Loss	Addresses class imbalance by down-weighting easy examples.	More computationally demanding than standard cross-entropy; Requires tuning of the focusing parameter.
Jaccard Loss	Directly optimizes overlap measures; Ideal for segmentation tasks.	Computationally intensive; Complex gradient calculations.
Adversarial Loss	Enhances realism in generated images; Models distribution well.	Requires simultaneous training of generator and discriminator; Can lead to instability.
Categorical Cross-Entropy Loss	Works well for multi-class problems; Provides probabilistic outputs.	Class imbalance issues observed; Similar to standard cross-entropy.
Contrastive Loss	Effective for learning similarities and differences between pairs.	Requires careful curation of pairs; Increases implementation complexity.
Diffusion Noise Prediction Loss	Useful in modern generative models; Handles noise explicitly.	High computational burden; Model complexities can complicate training.
Perceptual Loss	Captures perceptual differences; Leads to visually appealing results.	Computationally expensive due to feature extraction; May require significant model adjustments.
Variational Autoencoder (VAE) Loss	Provides a probabilistic framework for generative models.	Balancing reconstruction and KL divergence can be tricky; May need careful tuning of parameters.





HYPERPARAMETERS-MAGNITUDE HYPERPARAMETERS

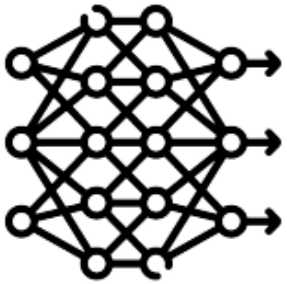
HYPERPARAMETERS-MAGNITUDE HYPERPARAMETERS

In machine learning, a hyperparameter is **a parameter whose value is used to control the learning process**. By contrast, the values of other parameters (typically node weights) are derived via training.

Hyperparameters



Parameters



HYPERPARAMETERS-MAGNITUDE HYPERPARAMETERS

- **Hyperparameters –**
  - Settings chosen *before* a machine learning model trains, controlling the learning process (like learning rate, batch size, epochs, network layers)
- **Magnitude Hyperparameters-**
  - Specific types that directly manage the *size or scale* of model weights, such as **weight decay** (to keep weights small) or **initialization** values,
  - Crucial for model performance, distinct from learned parameters (weights/biases) which are internal to the model.



HYPERPARAMETERS

General Hyperparameters

- Define the model's structure and training dynamics:
  - **Learning Rate**: Step size for parameter updates.
  - **Batch Size**: Number of samples processed before parameter update.
  - **Epochs**: Number of full passes through the training data.
  - **Network Architecture**: Number of layers, neurons, etc. (in neural networks).



HYPERPARAMETERS

- Momentum-
  - degree to which models update parameters in the same direction as previous iterations, rather than reversing course.
  - Begin with a lower hyperparameter value for momentum and then tweak upwards as needed to keep the model on course as it takes in training data.
  - Momentum helps the learning algorithm get out of spots in the search space where it would otherwise become stuck.
  - In the errorscape, it helps the updater find the gulleys that lead toward the minima.
  - Momentum is to the learning rate what the learning rate is to weights, and it helps us produce better quality models

- Epochs-

- Activation function



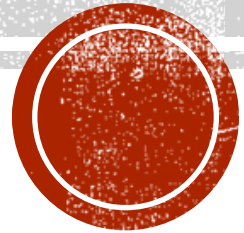
HYPERPARAMETERS-MAGNITUDE HYPERPARAMETERS

Magnitude Hyperparameters (Weight-Related)-

- Specifically deal with the scale of the model's internal weights, often in deep learning:
  - **Weight Decay (L2 Regularization):** Adds a penalty for large weights, encouraging smaller magnitudes to prevent overfitting.
  - **Initialization**: Values used to start weights, influencing their initial magnitude.
  - **Learning Rate & Decay**: Can directly control how quickly weight magnitudes change



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS



WHAT IS DEEP LEARNING?

- More neurons than previous networks
- More complex ways of connecting layers
- Automatic feature extraction



ARCHITECTURES OF DEEP NETWORKS

- Four major architectures of deep networks-
  - Unsupervised Pretrained Networks-
    - Autoencoders
    - Deep Belief Networks (DBNs)
    - Generative Adversarial Networks (GANs)
  - Convolutional Neural Networks
  - Recurrent Neural Networks
  - Recursive Neural Networks



ADVANCES IN NETWORK ARCHITECTURE

- Advances in layer types
- Advances in neuron types
- Hybrid architectures



FROM FEATURE ENGINEERING TO AUTOMATED FEATURE LEARNING

- Feature engineering-handcrafted features → highly accurate models but take a lot of time and experience to produce.
- Feature learning-
  - Feature maps
  - CNN



FROM FEATURE ENGINEERING TO AUTOMATED FEATURE LEARNING

Generative modelling-

- Inceptionism-
  - Technique in which a trained convolutional network is taken with its layers in reverse order and given an input image coupled with a prior constraint.
  - The images are modified iteratively to enhance the output in a manner that could be described as "hallucinative."
- Modeling artistic style-to learn the style of specific painters and then generate a new image in this style of arbitrary photographs
- Generative Adversarial Networks- Synthesizing novel images by modeling the distributions of input data seen by the network.
- Recurrent Neural Networks-



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Core components-

- Parameters
- Layers
- Activation functions
- Loss functions
- Optimization methods
- Hyperparameters



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Core components-

- Parameters-
  - Hyperparameters
  - Optimization methods
  - Different architectures for layer connection



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Core components-

- Layers-
  - Fundamental architectural unit
  - Combinations of layers /activation functions
  - Layer specific hyperparameters



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Core components-

- **Activation Functions-**
  - Two main areas across all architectures:
    - Hidden layers
    - Output layers
  - Hidden layer activation functions-
    - Sigmoid
    - Tanh
    - Hard tanh
    - Rectified linear unit (ReLU) (and its variants)
  - A more continuous distribution of input data is generally best modeled with a ReLU activation function.
  - Optionally use tanh activation function (if the network isn't very deep) in the event that the ReLU did not achieve good results



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Core components-

Activation Functions-

- Output layer for regression-
  - What type of answer we expect our model to output.
  - For e.g. a single real-valued number as o/p from our model, then use a linear activation function.
- Output layer for binary classification-
  - Use a sigmoid output layer with a single neuron to give us a real value in the range of 0.0 to 1.0 (excluding those values) for the single class.
  - This real-valued output is typically interpreted as a probability distribution.
- Output layer for multiclass classification-
  - If we only care about the best score across these classes, use a softmax output layer with an arg-max() function to get the highest score of all the classes.
  - The softmax output layer gives us a probability distribution over all the classes.



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Core components-

Loss functions-

- Regression
- Classification
- Reconstruction-
  - Involved in unsupervised feature extraction
  - In certain architectures of deep networks reconstruction loss functions help the network extract features more effectively when paired with the appropriate activation function.
  - Example - use the multiclass cross-entropy as a loss function in a layer with a softmax activation function for classification output



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Core components-

- **Optimization Algorithms-**
  - First-order
    - calculate the Jacobian matrix.
    - The Jacobian has one partial derivative per parameter
    - The algorithm then takes one step in the direction specified by the Jacobian.
    - Calculate a gradient (Jacobian) at each step to determine which direction to go in next.
    - At each iteration, or step, algorithm try to find the next best possible direction to go, as defined by objective function.
    - For e.g- GD

$$\mathbf{H}_f = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1\,\partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1\,\partial x_n} \\ \frac{\partial^2 f}{\partial x_2\,\partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2\,\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n\,\partial x_1} & \frac{\partial^2 f}{\partial x_n\,\partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix},$$



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Core components-

- **Optimization Algorithms-**
  - Second-order-
    - Calculate or approximate the Hessian
    - Calculate the derivative of the Jacobian (i.e., the derivative of a matrix of derivatives) by approximating the Hessian.
    - Take into account interdependencies between parameters when choosing how much to modify each parameter
    - Methods-
      - Limited-memory BFGS (L-BFGS) Broyden-Fletcher-Goldfarb-Shanno
      - Conjugate gradient
      - Hessian-free



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Core components-

- Hyperparameters-any configuration setting free to be chosen by the user that might affect performance.
- Categories-
  - Layer size
  - Magnitude (momentum, learning rate)
  - Regularization (dropout, drop connect, L1, L2)
  - Activations (and activation function families)
  - Weight initialization strategy
  - Loss functions
  - Settings for epochs during training (mini-batch size)
  - Normalization scheme for input data (vectorization)



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Hypermeters-

1. Layer size

- Number of neurons in a given layer.
- For the input layer- match up to the number of features in the input vector.
- For the output layer- either be a single output neuron or a number of neurons matching the number of classes we are trying to predict.
- Challenge in hyperparameter tuning- Deciding on neuron counts for each hidden layer
- Complexity of a problem -directly correlated to how many neurons to be used in the hidden layers of networks
- More parameters -increase in the amount of effort needed to train the network.
- Large parameter counts can lead to long training times and models that struggle to find convergence.



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Hypermeters-

2. Magnitude hyperparameters-

- Learning rate-
  - One of the key hyperparameters in neural networks.
  - How fast we change the parameter vector as we move through search space.
  - Too high--move toward goal faster (but we might also take a step so large that we shoot right past the best answer to the problem)
  - Too small-longer time for training process to complete, make our learning algorithm inefficient.
  - Specific to the dataset and to other hyperparameters.
  - Schedule learning rates to decrease over time according to some rule



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Hypermeters-

2. Magnitude hyperparameters-

- Sparsity –
  - Recognizes that for some inputs only a few features are relevant

- Momentum-
  - Momentum helps the learning algorithm get out of spots in the search space where it would otherwise become stuck.
  - Momentum is to the learning rate what the learning rate is to weights, and it helps us produce better quality models



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Hypermeters-

2. Magnitude hyperparameters-

- **Momentum-**
  - **Nesterov's momentum (Nesterov Accelerated Gradient (NAG)-**
    - Momentum is a factor between 0.0 and 1.0 that is applied to the rate of change of the weights over time.
    - Typical value for momentum between 0.9 and 0.99.
  - **AdaGrad**
    - One technique developed to help augment finding the "right" learning rate.
    - Named in reference to how it "adaptively" uses sub-gradient methods to dynamically control the learning rate of an optimization algorithm.
    - Monotonically decreasing and never increases the learning rate above whatever the base learning rate was set at initially.
    - Speeds training in the beginning and slows it appropriately toward convergence, allowing for a smoother training process.



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

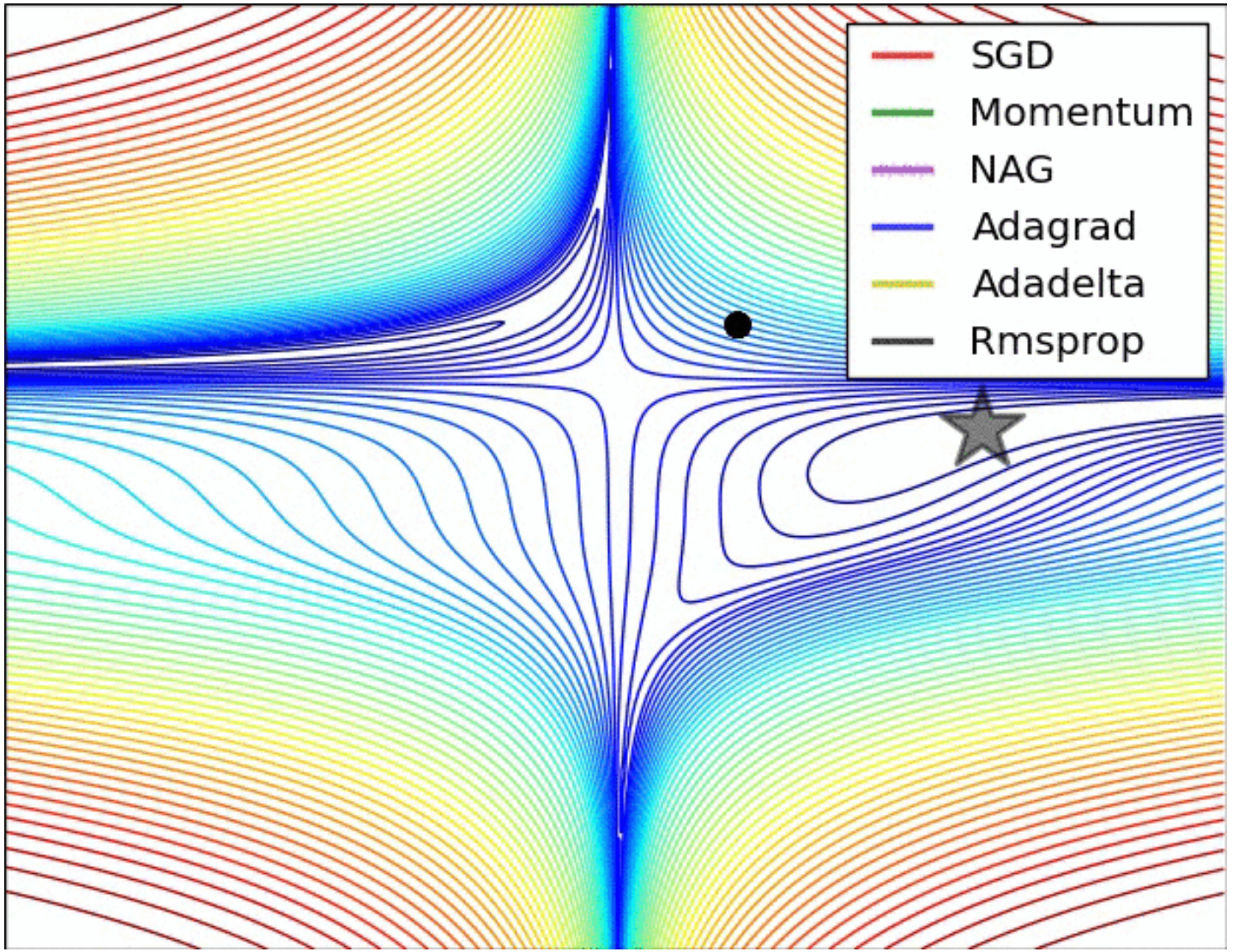
Hypermeters-

2. Magnitude hyperparameters-

- **Momentum-**
  - **RMSProp-**
    - Very effective, but currently unpublished adaptive learning rate method
  - **Adam-**
    - ADAM (a more recently developed updating technique from the University of Toronto) derives learning rates from estimates of first and second moments of the gradients
  - **AdaDelta-**
    - Variant of AdaGrad that keeps only the most recent history rather than accumulating it like AdaGrad does.



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS



Several optimizers as applied to Beale's function. The gray star indicates the location of the global minimum. Source: Alec Radford's post on visualizing optimization algorithms.

COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Regularization-

- Measure taken against overfitting

- Regularization for hyperparameters helps modify the gradient so that it doesn't step in directions that lead it to overfit
  - Dropout
  - DropConnect
  - L1 penalty
  - L2 penalty

- Regularization works by adding an extra term to the normal gradient computed.



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Regularization-

- *Dropout-*
  - Mechanism used to improve the training of neural networks by omitting a hidden unit.
  - Speeds training.
  - Driven by randomly dropping a neuron so that it will not contribute to the forward pass and backpropagation.
- *DropConnect*
  - Does the same thing as Dropout, but instead of choosing a hidden unit, it mutes the connection between two neurons.
- **Dropout and DropConnect mute parts of the input to each layer, such that the neural network learns other portions.**
- **Zeroing-out parts of the data causes a neural network to learn more general representations**



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Regularization-

- The penalty methods L1 and L2, in contrast, are a way of preventing the neural network parameter space from getting too big in one direction.
- They make large weights smaller.

- *L1-*
  - Computationally inefficient in the nonsparse case, has sparse outputs, and includes built-in feature selection.
  - Multiplies the absolute value of weights rather than their squares.
  - This function drives many weights to zero while allowing a few to grow large, making it easier to interpret the weights



COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Regularization-

- L2 –
  - Computationally efficient due to analytical solutions and nonsparse outputs, but it does not do feature selection automatically.
  - Common and simple hyperparameter, adds a term to the objective function that decreases the squared weights.
  - Multiply half the sum of the squared weights by a coefficient called the weight-cost.
  - Improves generalization, smooths the output of the model as input changes, and helps the network ignore weights it does not use.



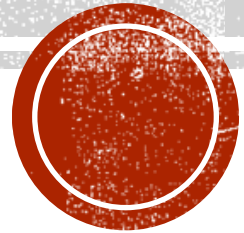
COMMON ARCHITECTURAL PRINCIPLES OF DEEP NETWORKS

Mini-batching-

- Send more than one input vector (a group or batch of vectors) to be trained in the learning system.
- Also allows us to compute certain linear algebra operations (specifically matrix-to-matrix multiplications) in a vectorized fashion.
- Option of sending the vectorized computations to GPUs if they are present.



BUILDING BLOCKS OF DEEP NETWORKS



BUILDING BLOCKS OF DEEP NETWORKS

Specific building blocks

- Feed-forward multilayer neural networks
- Restricted Boltzmann Machines (RBMs)
- Autoencoders

- Both RBMs and autoencoders are characterized by an extra layer-wise step for training.
- Often used for the pretraining phase in other larger deep networks.



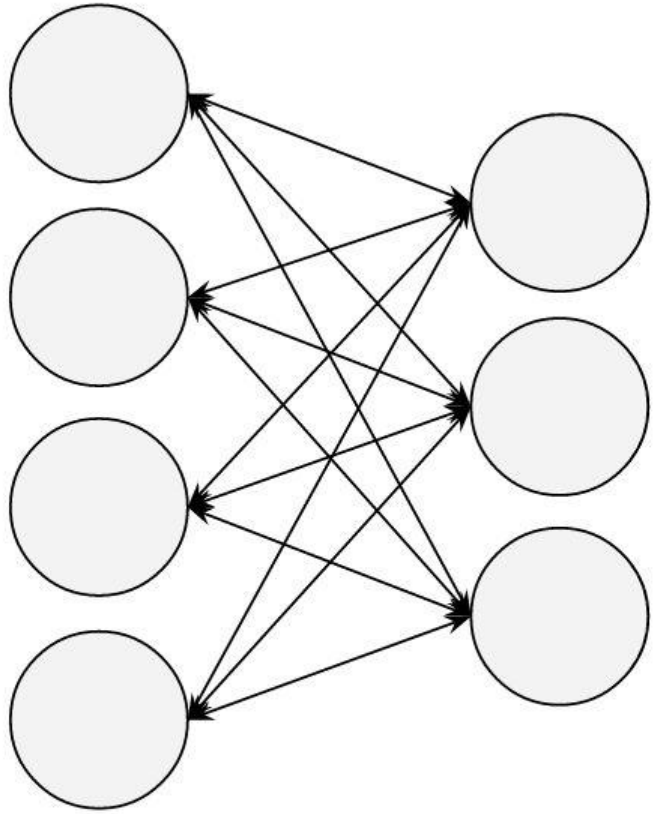
BUILDING BLOCKS OF DEEP NETWORKS

- **RBMs-**
  - Model probability and are great at feature extraction.
  - Feedforward networks in which data is fed through them in one direction with two biases rather than one bias as in traditional backpropagation feed-forward networks.
  - Used in deep learning for the following:
    - Feature extraction
    - Dimensionality reduction
- The “restricted” part of the name “Restricted Boltzmann Machines” - connections between nodes of the same layer are prohibited (e.g., there are no visible-visible or hidden-hidden connections along which signal passes).
- Geoff Hinton—” A network of symmetrically connected, neuron-like units that make stochastic decisions about whether to be on or off.”



BUILDING BLOCKS OF DEEP NETWORKS

- **RBMs-**
  - Also, a type of autoencoder
  - Used for pretraining layers in larger networks such as Deep Belief Networks.
- **Network layout**
- 5 main parts of a basic RBM:
  - Visible units
  - Hidden units
  - Weights
  - Visible bias units
  - Hidden bias units
- A standard RBM has a visible layer and a hidden layer
- Every visible unit is connected to every hidden unit,
- Each layer of an RBM can be imagined as a row of nodes.
- The nodes of the visible and hidden layers are connected by connections with associated weights



BUILDING BLOCKS OF DEEP NETWORKS

RBMs-

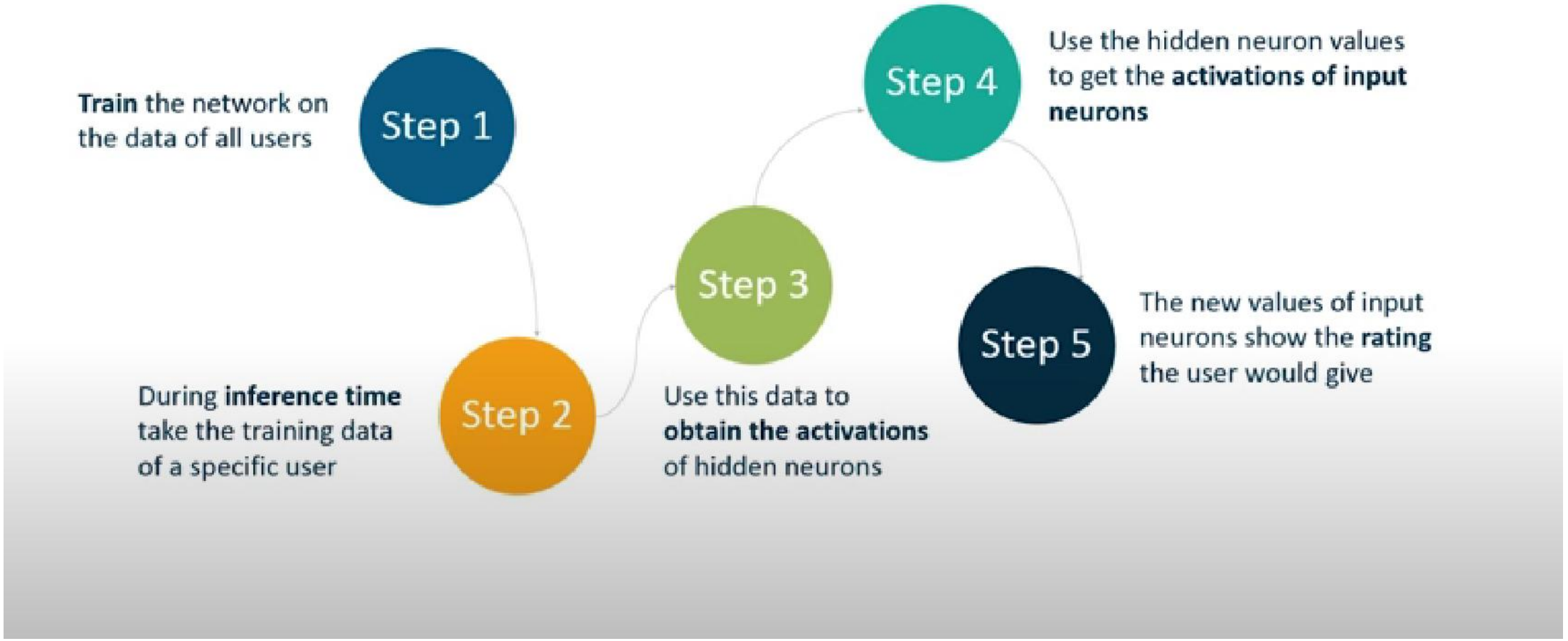
- Feed Forward Pass
- Feed Backward Equation:
  - **Error** = Reconstructed Input Layer-Actual Input layer
  - **Adjust Weight** = Input*error*learning rate (0.1)

- **Network layout**
  - *Visible and hidden layers-*
  - *Connections and weights-*
  - *Biases-*
- **Training-**
- **Reconstruction**



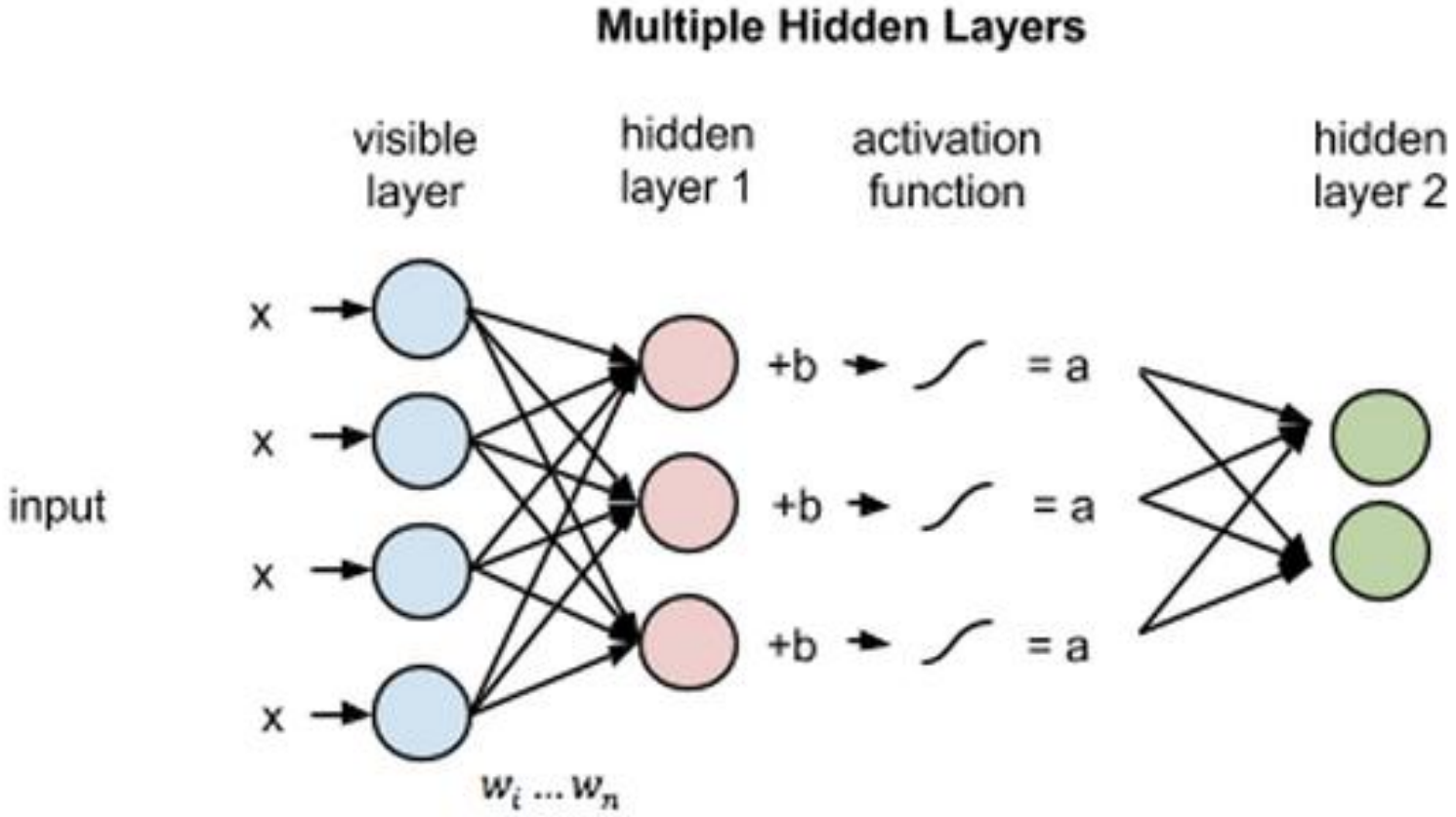
BUILDING BLOCKS OF DEEP NETWORKS

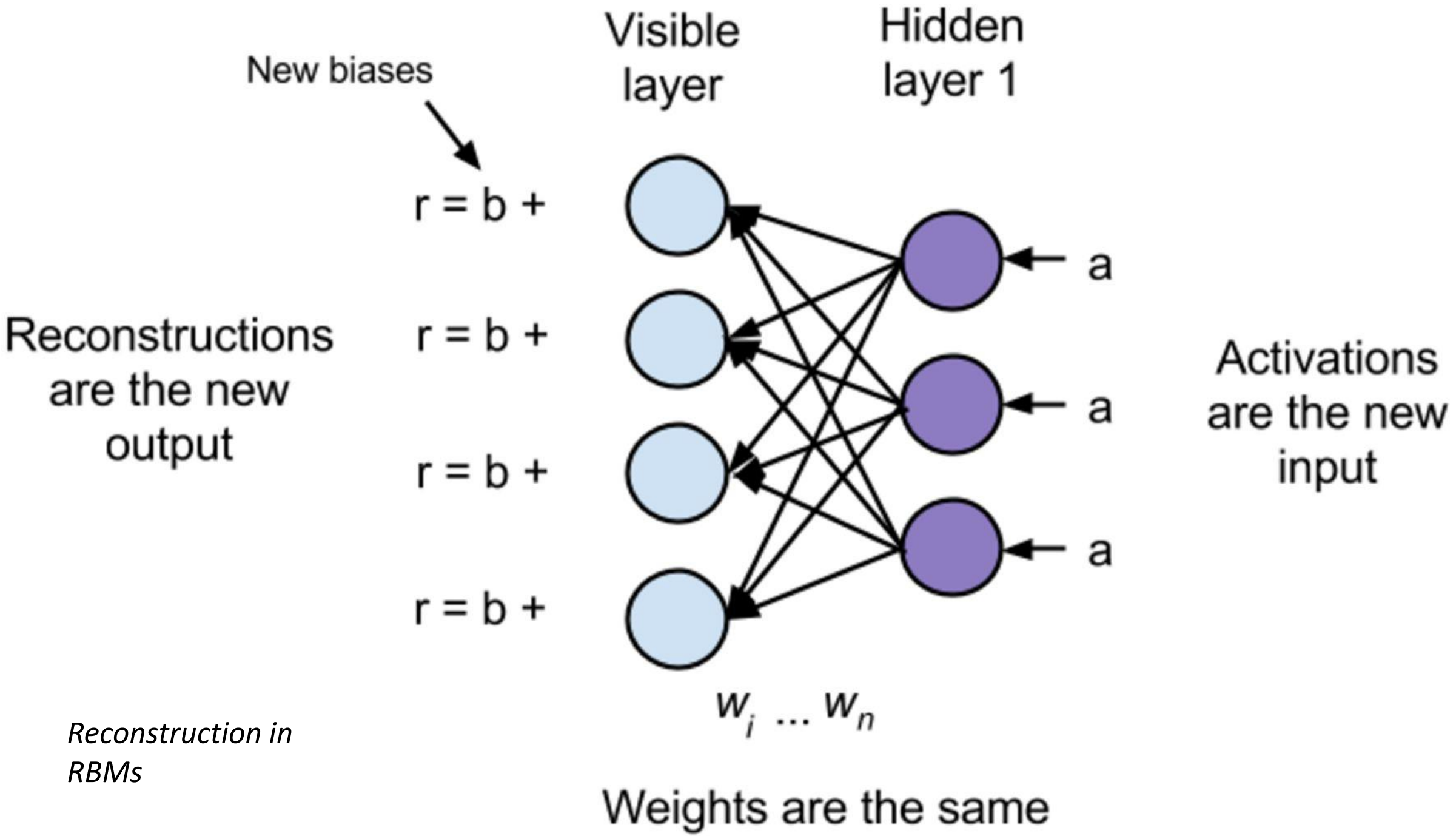
RBMS-



BUILDING BLOCKS OF DEEP NETWORKS

- **RBMs-**

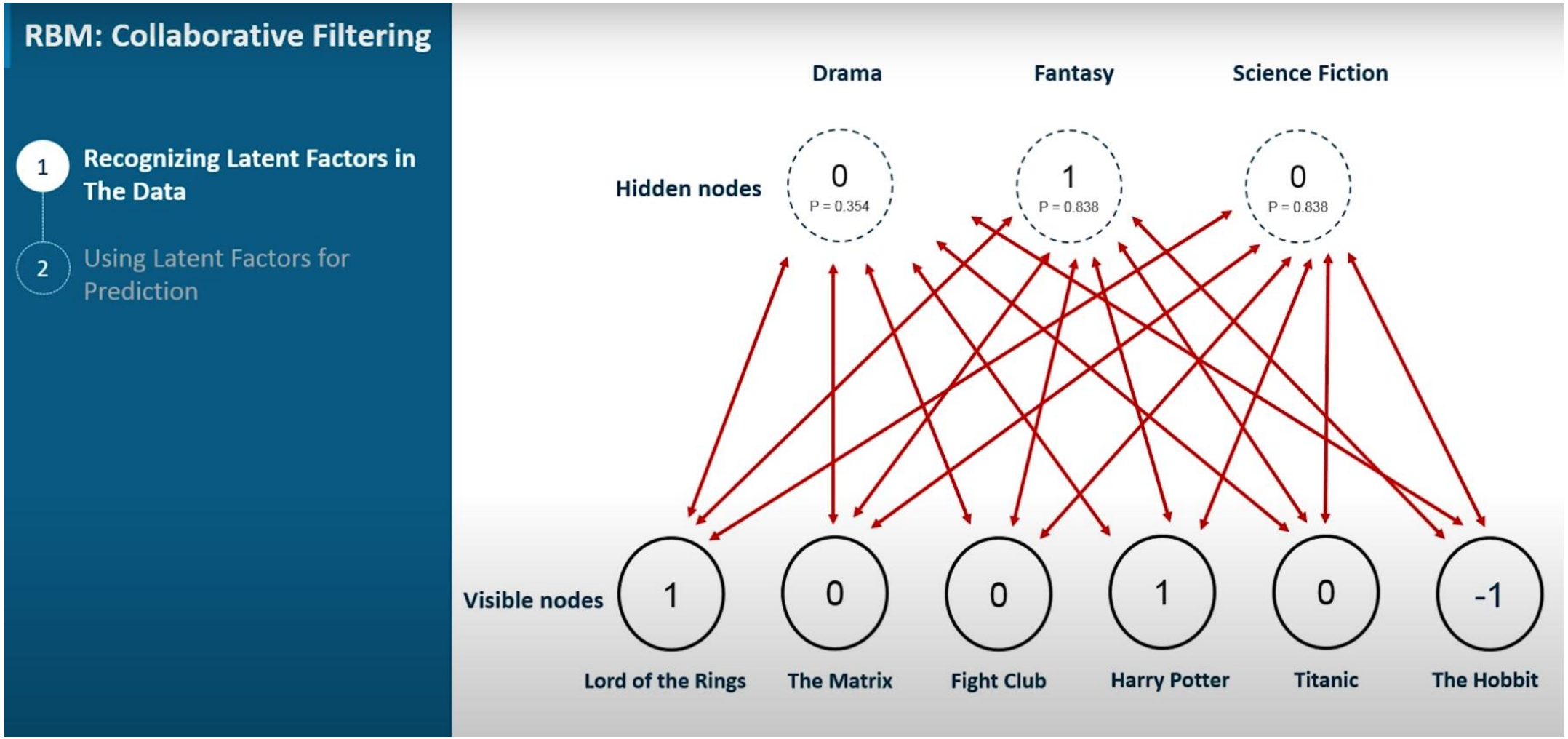




Reconstruction in RBMs

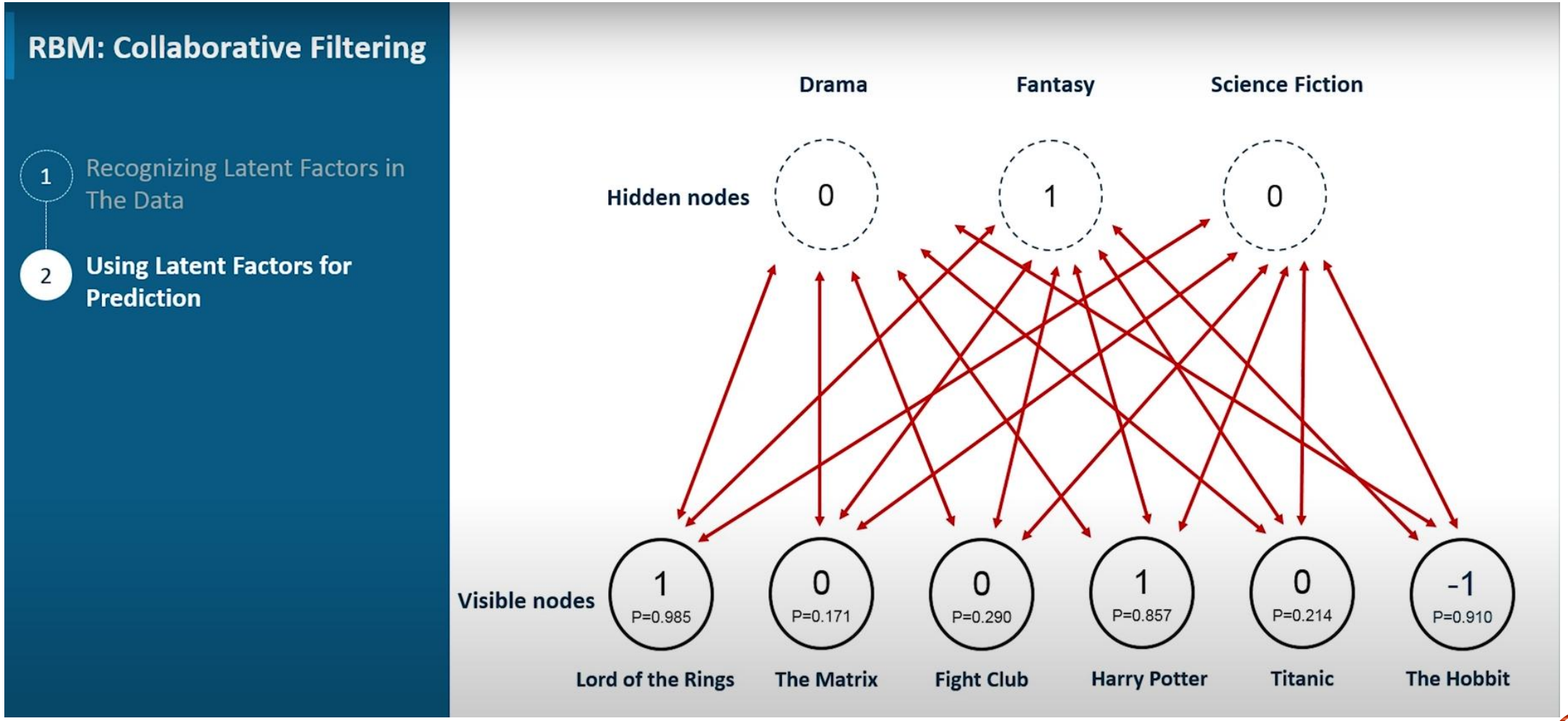
BUILDING BLOCKS OF DEEP NETWORKS

RBMS-



BUILDING BLOCKS OF DEEP NETWORKS

RBMS-



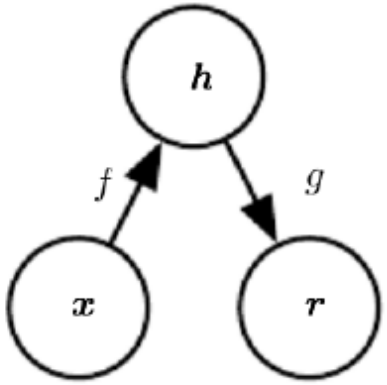
BUILDING BLOCKS OF DEEP NETWORKS

- **Other uses of RBMs-**
  - Dimensionality reduction
  - Classification
  - Regression
  - Collaborative filtering
  - Topic modelling
  - Imbalance data problem
  - Unstructured data
  - Noisy label problems



BUILDING BLOCKS OF DEEP NETWORKS

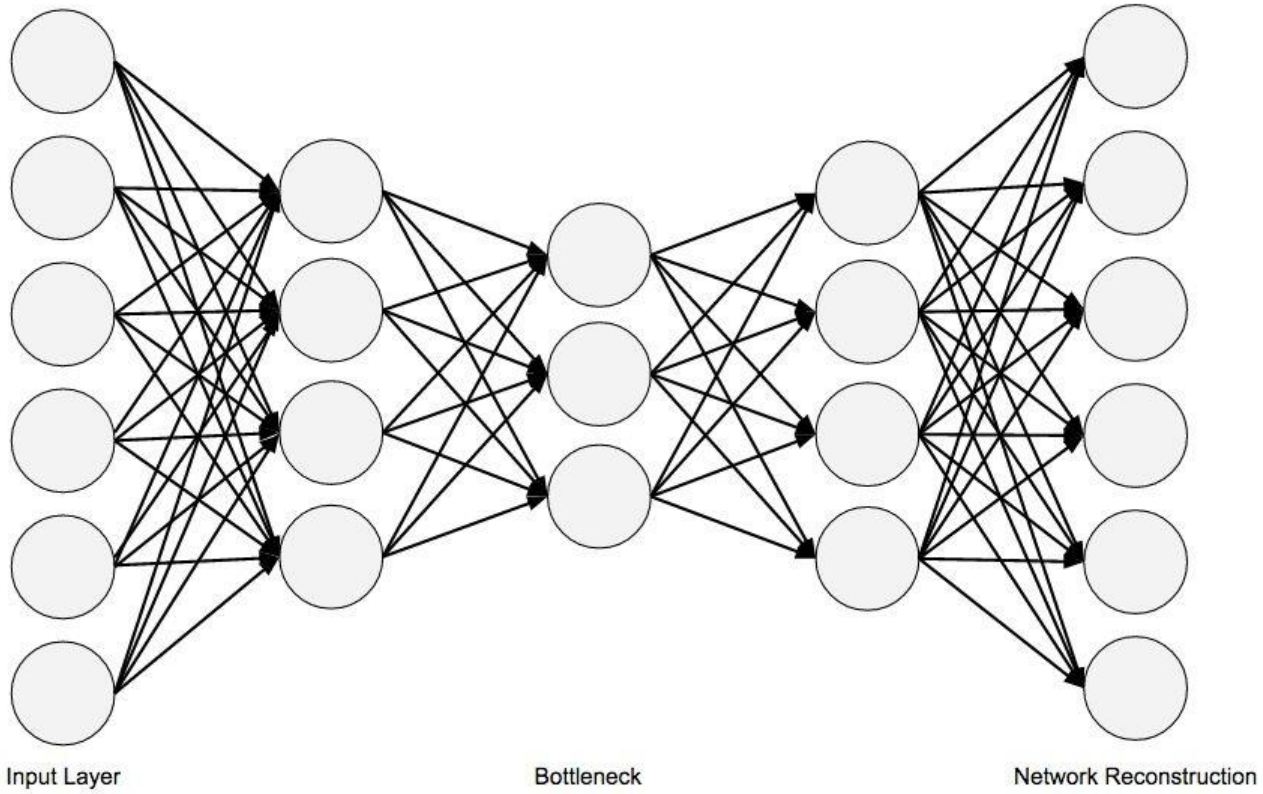
- Autoencoder is a neural network, which is trained to reproduce its input.
  - Encoder: $h = f(x)$
  - Decoder: $r = g(h)$



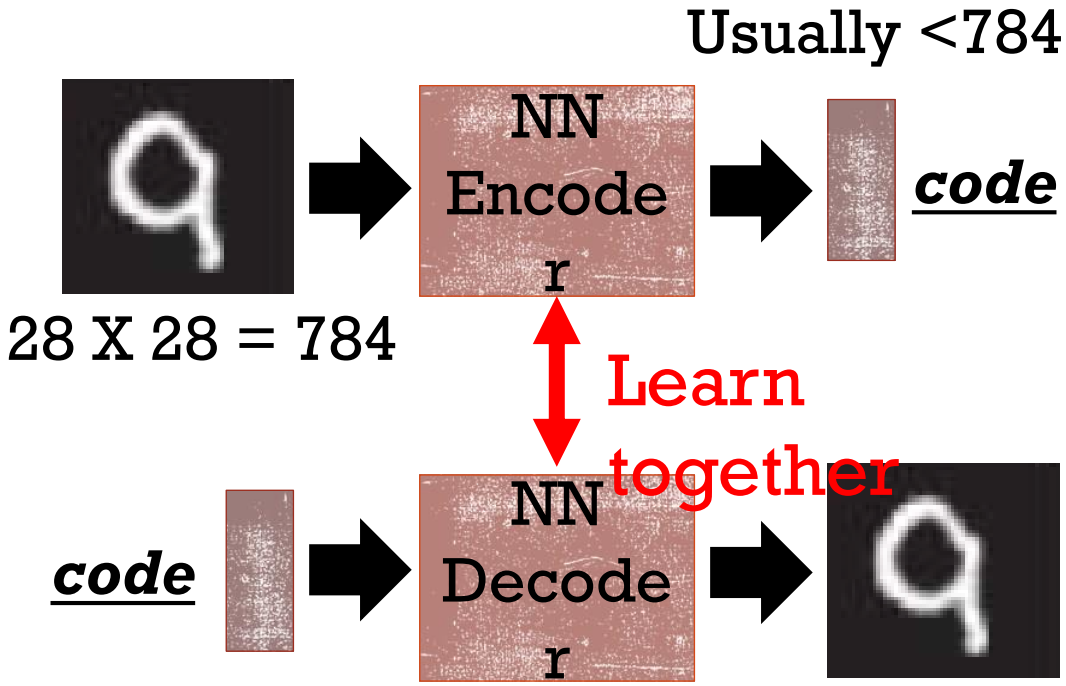
BUILDING BLOCKS OF DEEP NETWORKS

Autoencoders

- Used to learn compressed representations of datasets.
- To reduce a dataset's dimensionality.
- The output of the autoencoder network is a reconstruction of the input data in the most efficient form.
- **Similarities to multilayer perceptrons**
- Input layer, hidden layers of neurons, and then an output layer.
- Key diff- output layer in an autoencoder has the same number of units as the input layer does



AUTO-ENCODER



Compact representation of the input object

Can reconstruct the original object

DEEP AUTO-ENCODER

Original Image

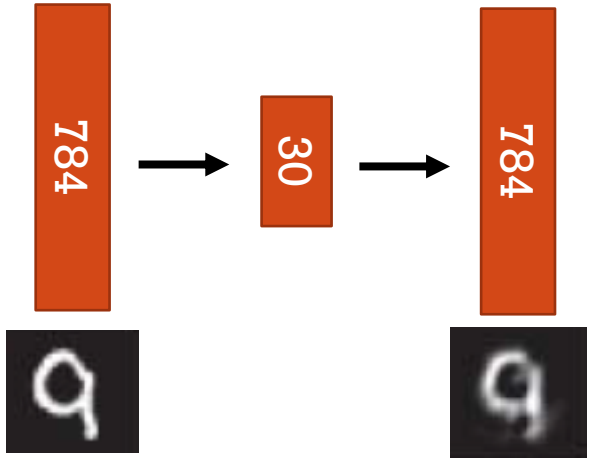


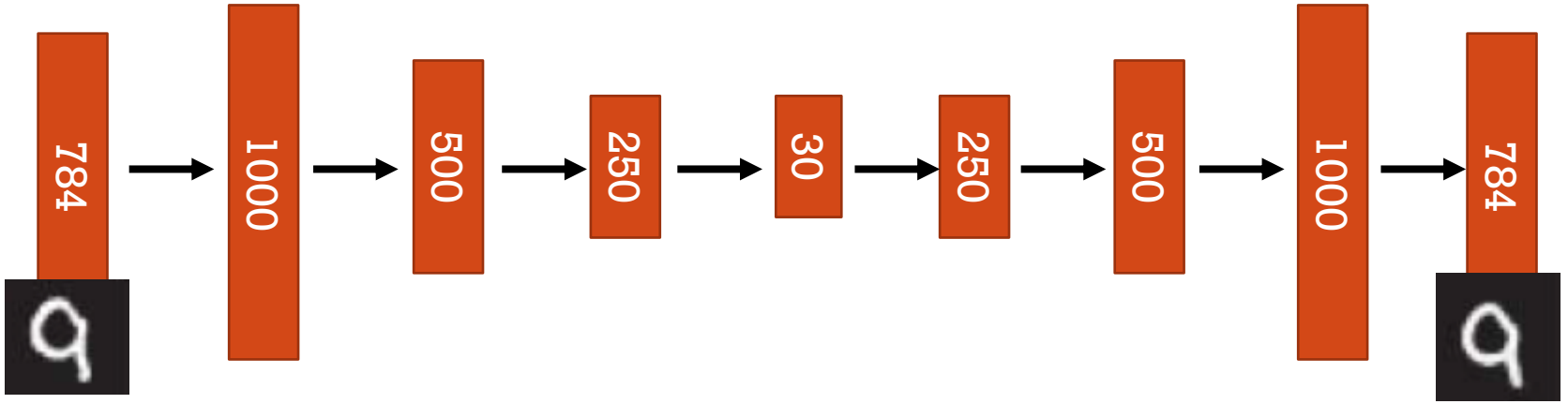
PCA



Deep Auto-encoder







BUILDING BLOCKS OF DEEP NETWORKS

Autoencoders-

Defining features of autoencoders

- Autoencoders differ from multilayer perceptrons in a couple of ways:
  - They use unlabeled data in unsupervised learning.
  - They build a compressed representation of the input data.
- ***Unsupervised learning of unlabeled data***
  - The autoencoder learns directly from unlabeled data.
  - Connected to the second major difference between multilayer perceptrons and autoencoders.
- ***Learning to reproduce the input data***
  - The goal of a multilayer perceptron network is to generate predictions over a class (e.g., fraud versus not fraud).
  - An autoencoder is trained to reproduce its own input data.



BUILDING BLOCKS OF DEEP NETWORKS

Autoencoders-

- **Training autoencoders**
- Autoencoders rely on backpropagation to update their weights.
- The main difference between RBMs and the more general class of autoencoders is in how they calculate the gradients



BUILDING BLOCKS OF DEEP NETWORKS

Common variants of autoencoders

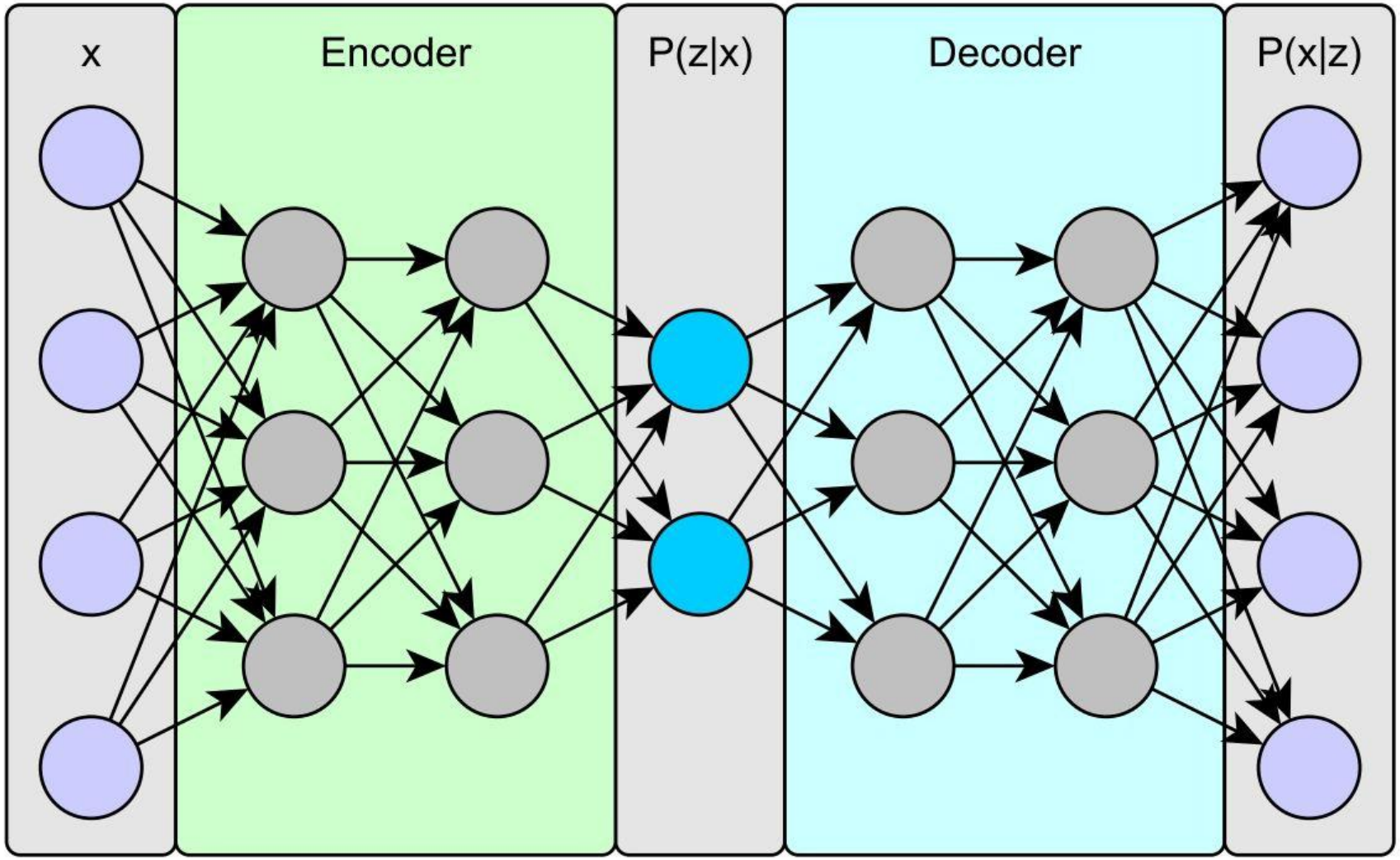
- *Compression autoencoders*
- *Denoising autoencoders*.

- *Compression autoencoders*
  - The network input must pass through a bottleneck region of the network before being expanded back into the output representation.

- *Denoising autoencoders*
  - Autoencoder is given a corrupted version (e.g., some features are removed randomly) of the input and the network is forced to learn the uncorrupted output.



BUILDING BLOCKS OF DEEP NETWORKS



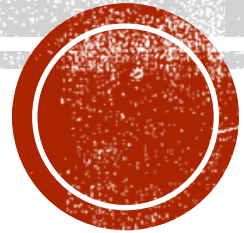
VAE network architecture

BUILDING BLOCKS OF DEEP NETWORKS

- Applications:
  - dimensionality reduction
    - visualization
    - feature extraction
    - ↑ prediction accuracy
    - ↑ speed of prediction
    - ↓ memory requirements
  - semantic hashing
  - unsupervised pretraining



MAJOR ARCHITECTURES OF DEEP NETWORKS



Dr. Sujata Pathak, IT, KJSSE

MAJOR ARCHITECTURES OF DEEP NETWORKS

- Unsupervised Pretrained Networks (UPNs)
- Convolutional Neural Networks (CNNs)
- Recurrent Neural Networks
- Recursive Neural Networks



UNSUPERVISED PRETRAINED NETWORKS

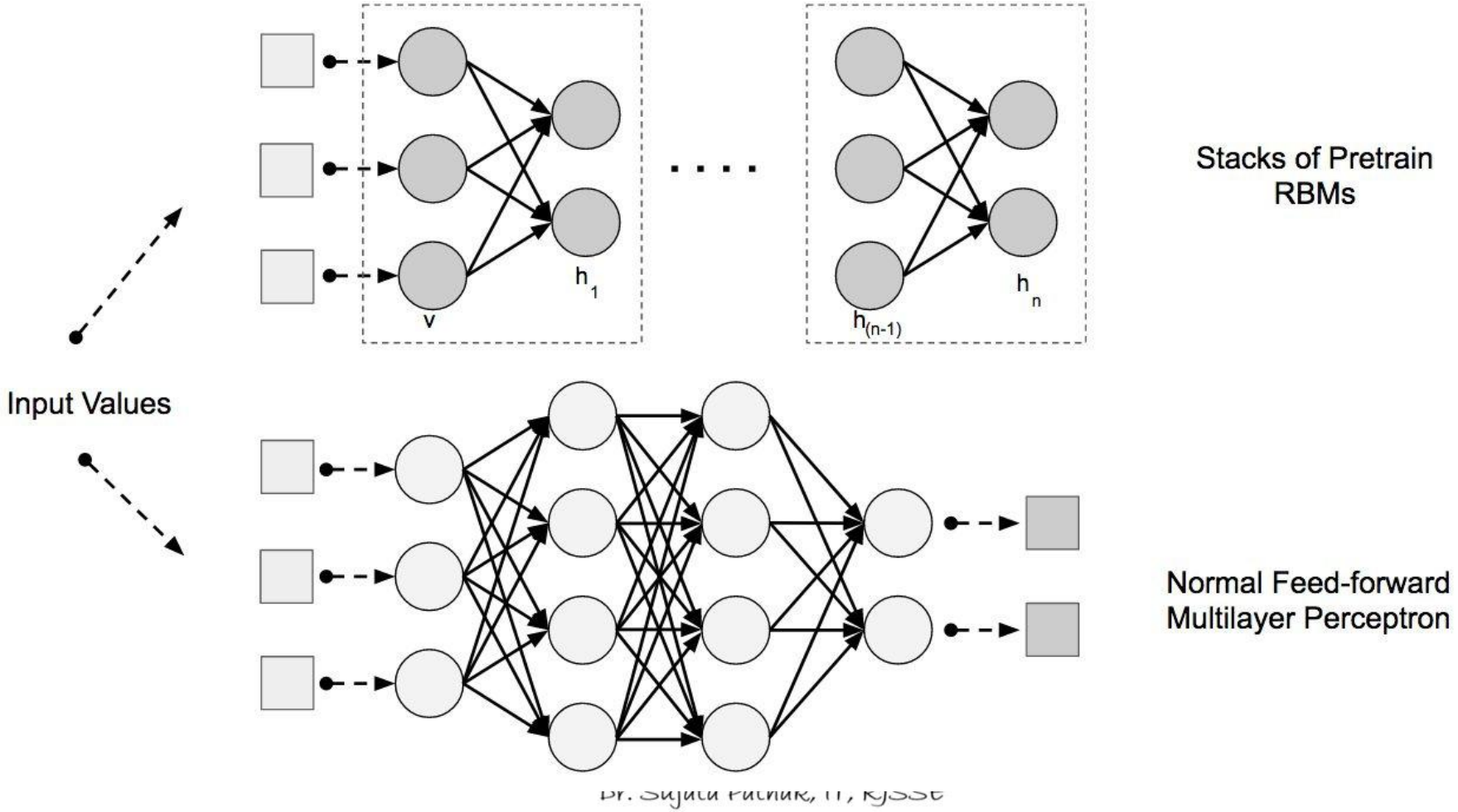
Specific architectures:

- Autoencoders
- Deep Belief Networks (DBNs)
- Generative Adversarial Networks (GANs)



DEEP BELIEF NETWORKS

- Composed of layers of Restricted Boltzmann Machines (RBMs) for the pretrain phase and then a feed-forward network for the fine-tune phase



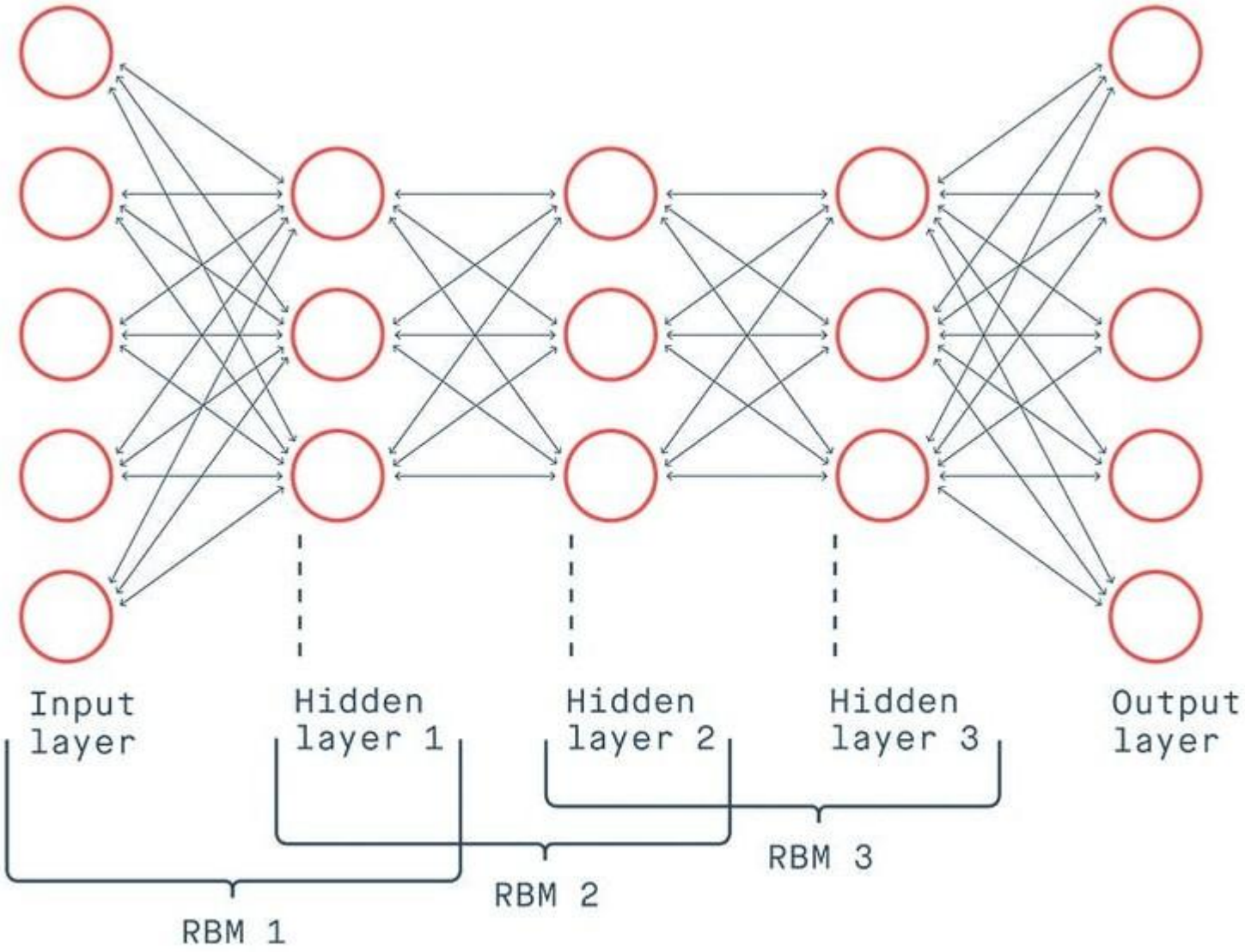
DEEP BELIEF NETWORKS

- Backpropagation-local minima
- Solution-Pre-training
- Pre-training-Back propagation-reduced error rate



DEEP BELIEF NETWORKS

- Network structure- DBN identical to MLP
- Training- DBN differs with MLP
- Role of Hidden nodes



DEEP BELIEF NETWORKS

Training-

- Greedy learning algorithm- from bottom layer moving upside, fine tuning generative weights
- Greedy learning algorithms- Quick and efficient, Optimize weights at each layer
- Each layer learns entire input
- DBN work globally and regulate each layer in order, as the model slowly improves



HOW DEEP BELIEF NETWORKS WORK?

- DBNs work in two main phases:
  - Pre-training
  - Fine-tuning.
- Pre-training phase-
  - Network learns to represent the input data layer by layer.
  - Each layer is trained independently as an RBM, which allows the network to learn complex data representations efficiently.
  - Network learns the probability distribution of the inputs, which helps it understand the underlying structure of the data.



HOW DEEP BELIEF NETWORKS WORK?

- Fine-tuning phase-
  - DBN adjusts its parameters for a specific task, like classification or regression.
  - Done using backpropagation, where the network's performance on a task is evaluated, and the errors are used to update the network's parameters.
  - Involves supervised learning, where the network is trained with labelled data.



DEEP BELIEF NETWORKS

Applications-

- *Image recognition*
- *Video recognition*
- *Motion capture data*
- *Image classification*
- *Text classification*
- *Image generation*
- *Speech recognition*

https://www.cs.toronto.edu/~hinton/digitmovie2.gif

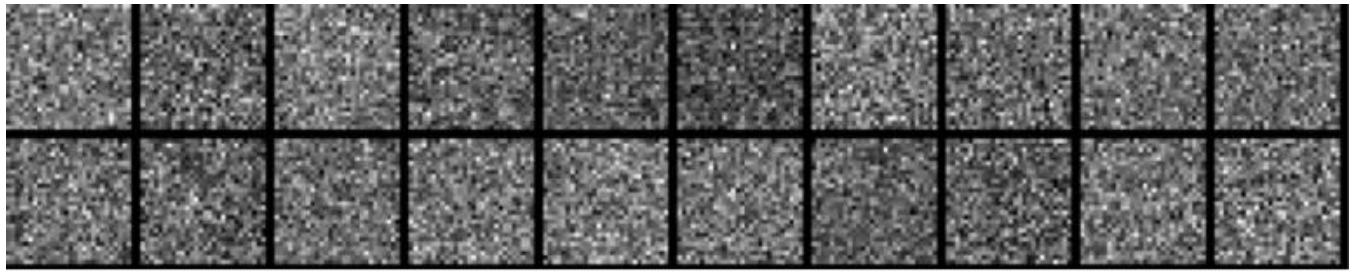


DEEP BELIEF NETWORKS

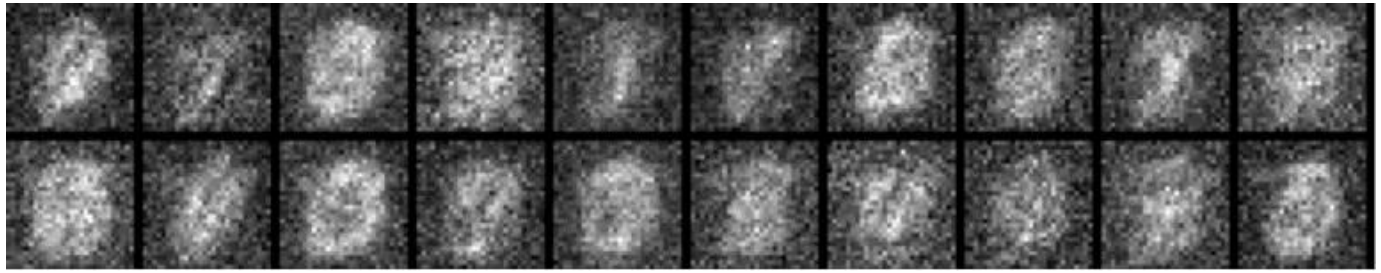
- **Feature Extraction with RBM Layers-**
  - RBMs -to extract higher-level features from the raw input vectors.
  - The fundamental purpose of RBMs in the context of deep learning and DBNs -to learn these higher-level features of a dataset in an unsupervised training fashion.
  - ***Learning higher-order features automatically-***
  - ***Initializing the feed-forward network-****fine-tune phase* of DBNs



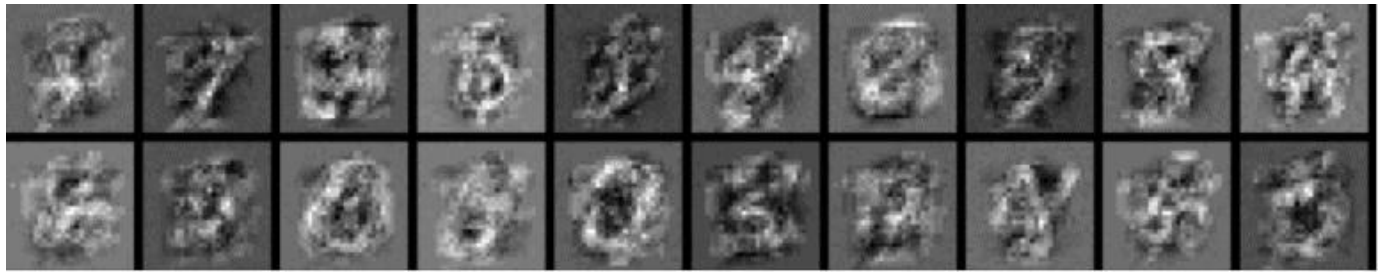
DEEP BELIEF NETWORKS



Activation render at the beginning of training

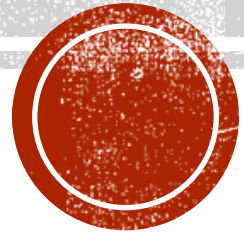


Features emerge in later activation render Figure



Portions of MNIST digits emerge towards end of training

GENERATIVE ADVERSARIAL NETWORKS (GAN)



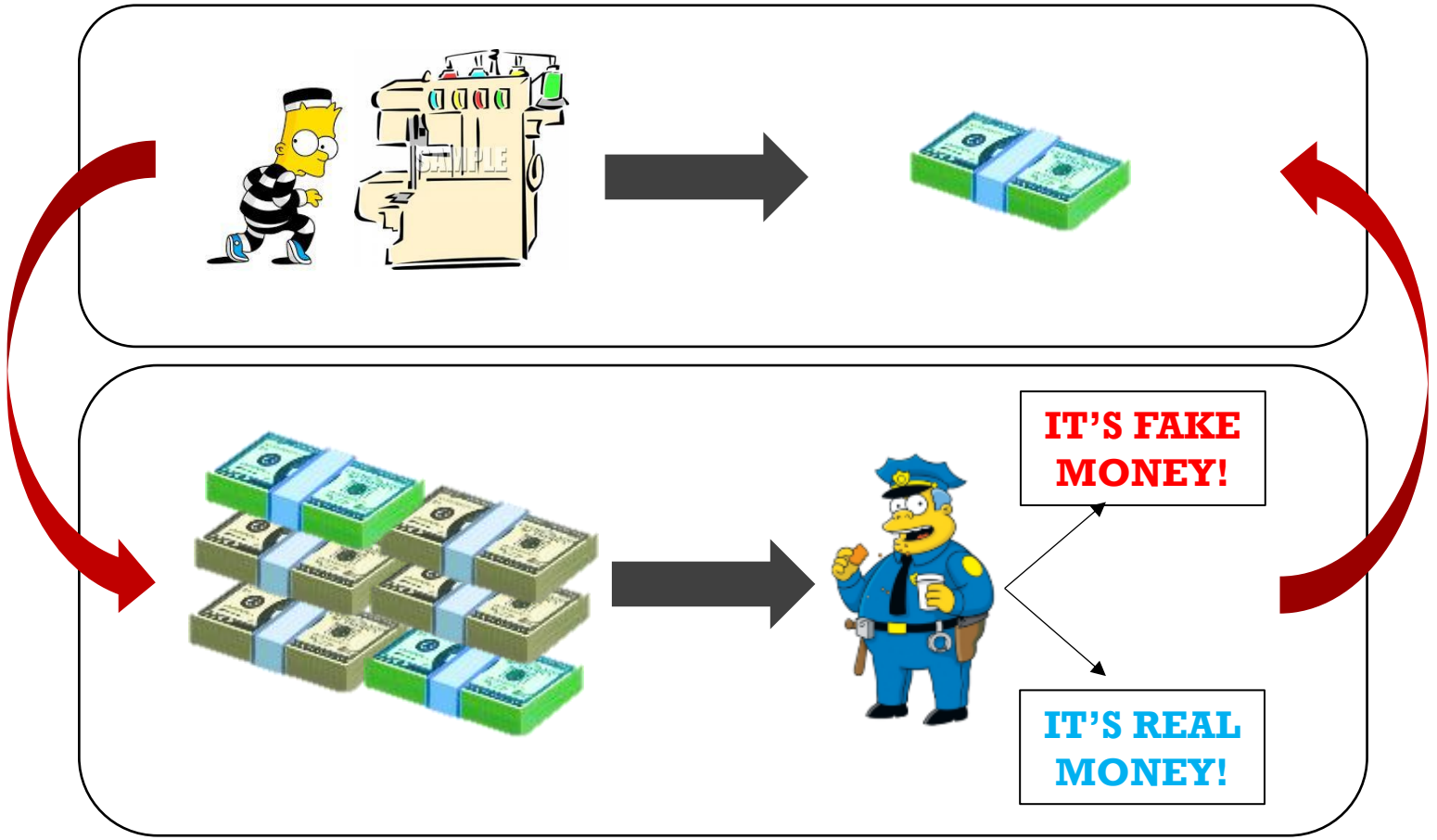
WHAT ARE GANS?

- System of two neural networks competing against each other in a zero-sum game framework.
- They were first introduced by Ian Goodfellow *et al.* in 2014.
- Can learn to draw samples from a model that is similar to data that we give them.



The intuition behind GAN: Counterfeiters vs Police Game

- **The generative model can be thought of as analogous to a team of counterfeiters, trying to produce fake currency, while the discriminative model is analogous to the police, trying to detect the counterfeit currency. Competition in this game drives both teams to improve their methods until the counterfeits are indistinguishable from the genuine articles.**



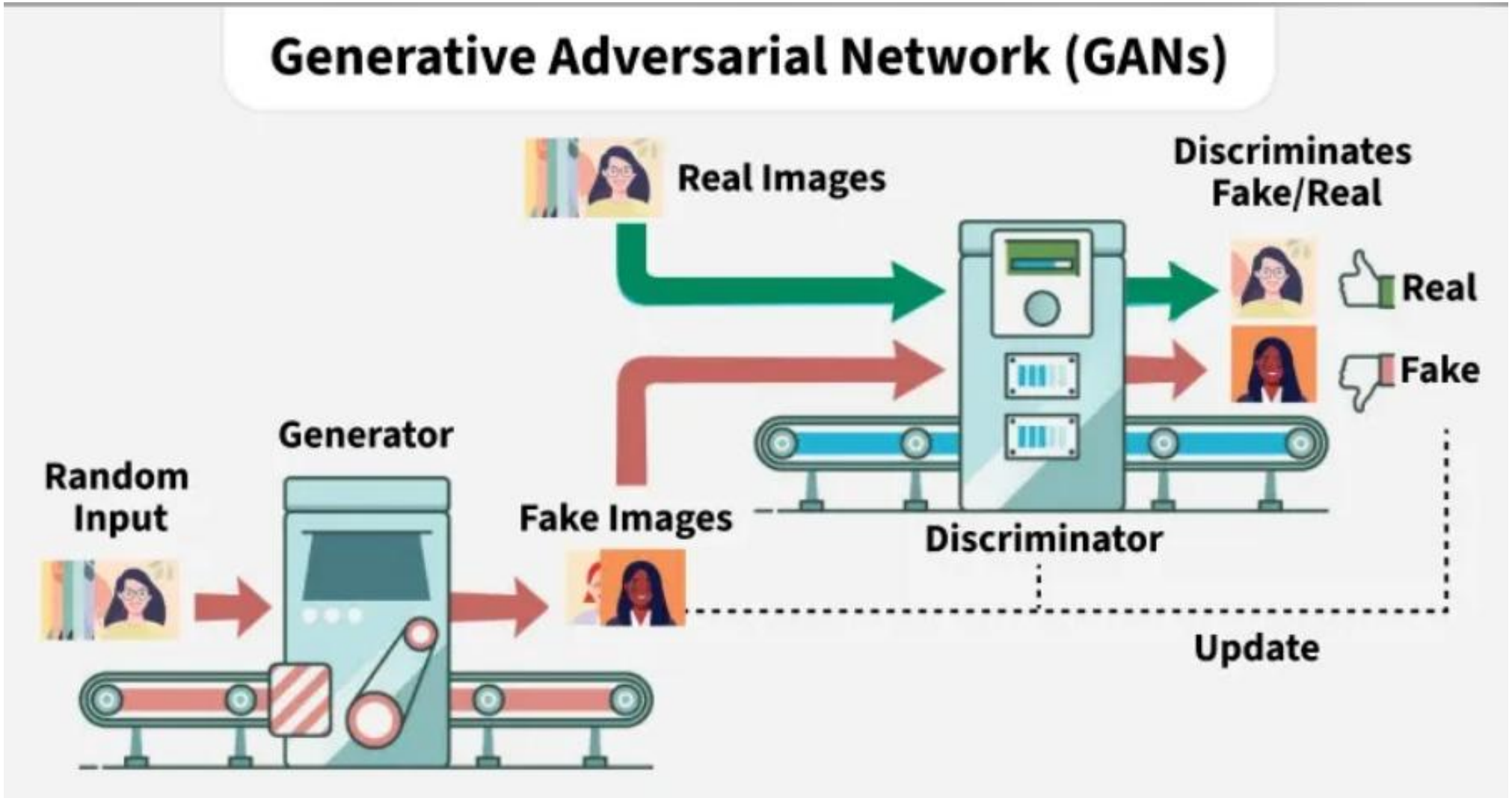
-Goodfellow et. al., "Generative Adversarial Networks" (2014)

What are GANs?

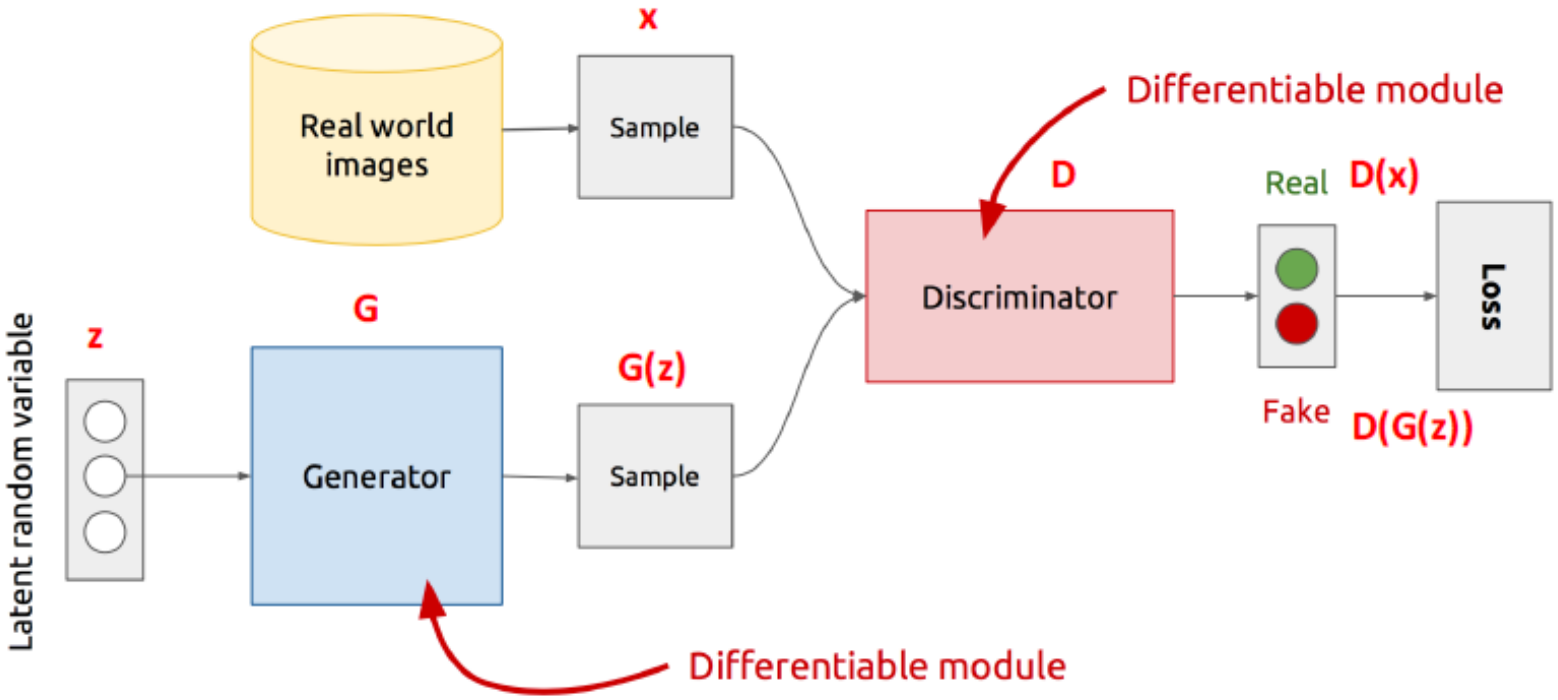
- GAN is a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models:
  - a generative model G that captures the data distribution.
  - a discriminative model D that estimates the probability that a sample came from the training data rather than G.
- The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game.
- In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation.



GAN’S ARCHITECTURE



GAN'S ARCHITECTURE



- **Z** is some random noise (Gaussian/Uniform).
- **Z** can be thought as the latent representation of the image.

https://www.slideshare.net/xavigiro/deep-learning-for-computer-vision-generative-models-and-adversarial-training-upc-2016



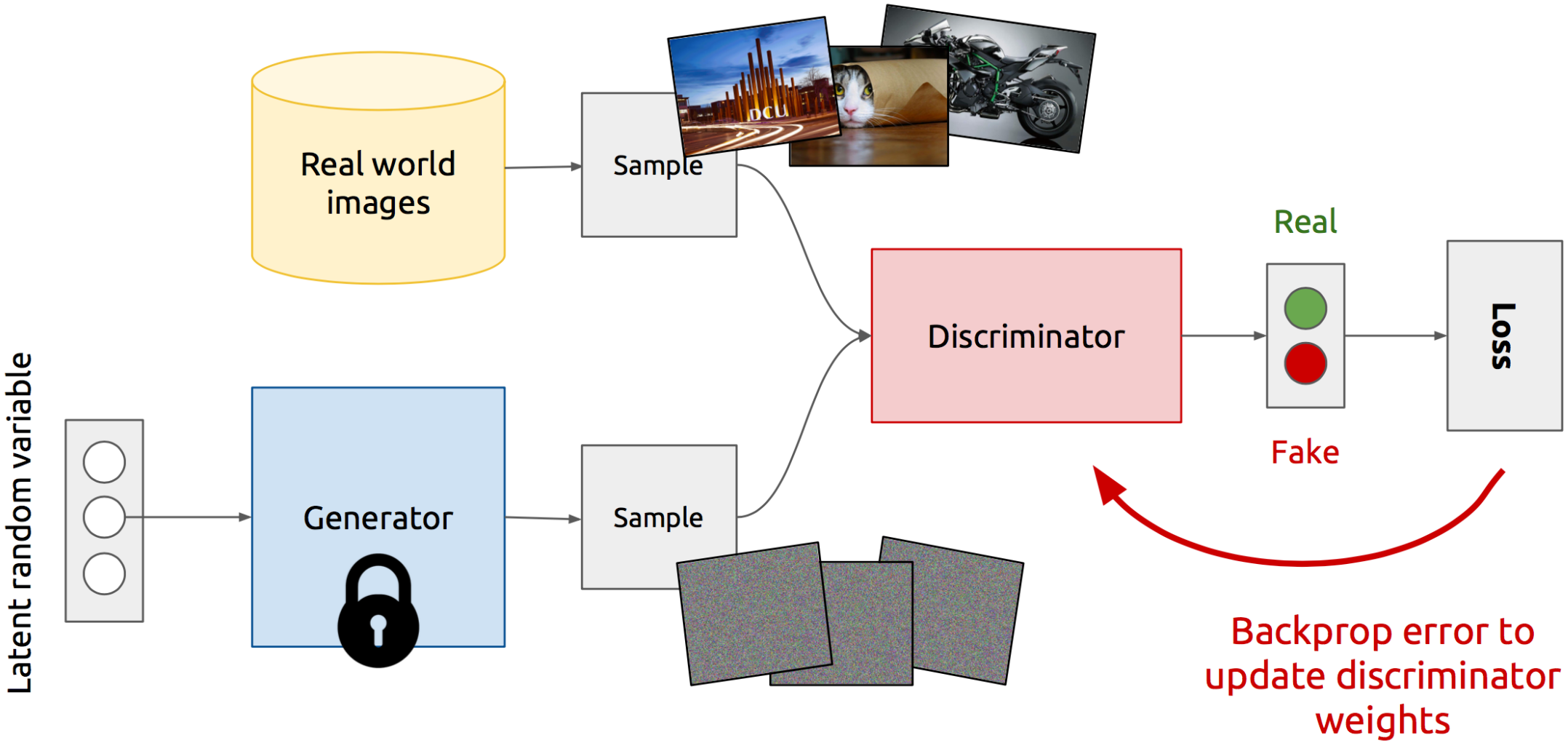
DISCRIMINATIVE MODELS

- A **discriminative** model learns a function that maps the input data (x) to some desired output class label (y).

- In probabilistic terms, they directly learn the conditional distribution $P(y|x)$.



TRAINING DISCRIMINATOR



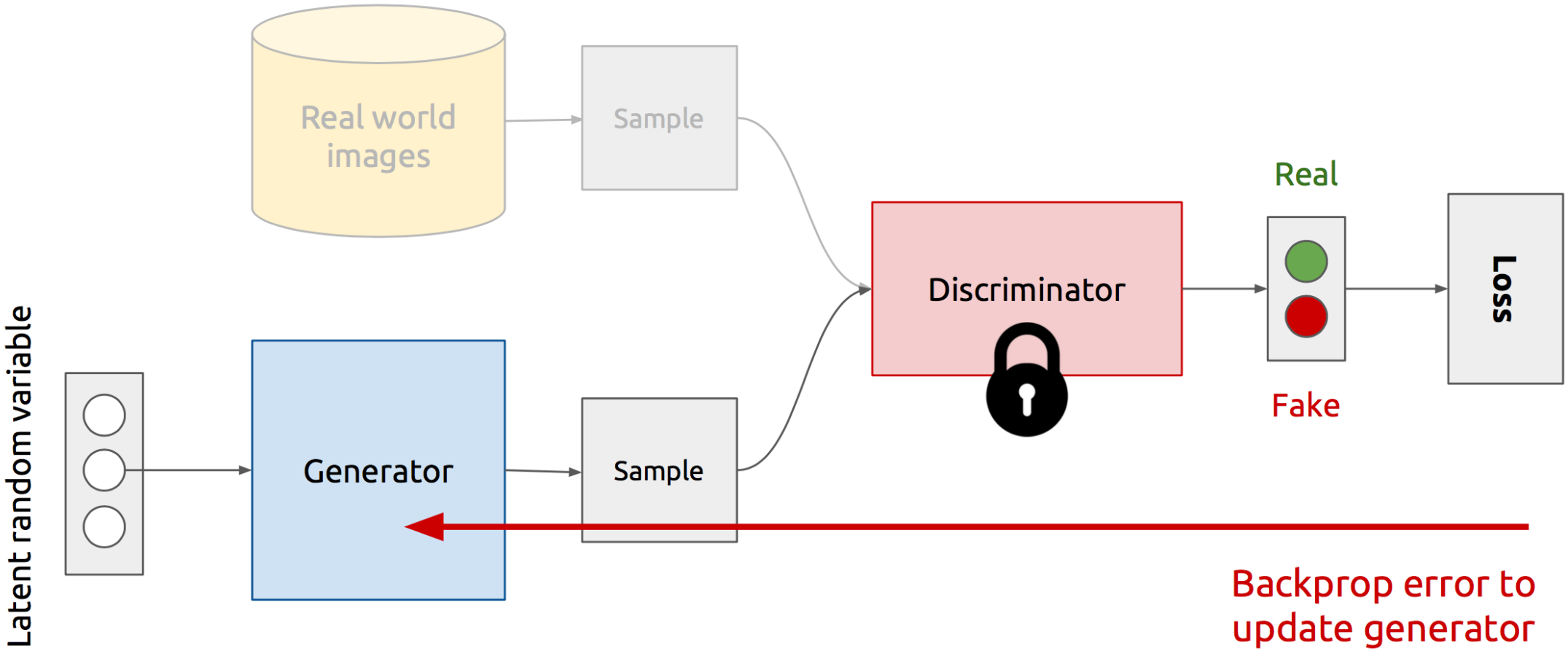
GENERATIVE MODELS

- A **generative** model tries to learn the joint probability of the input data and labels simultaneously i.e. $P(x,y)$.

- Potential to understand and explain the underlying structure of the input data even when there are no labels.



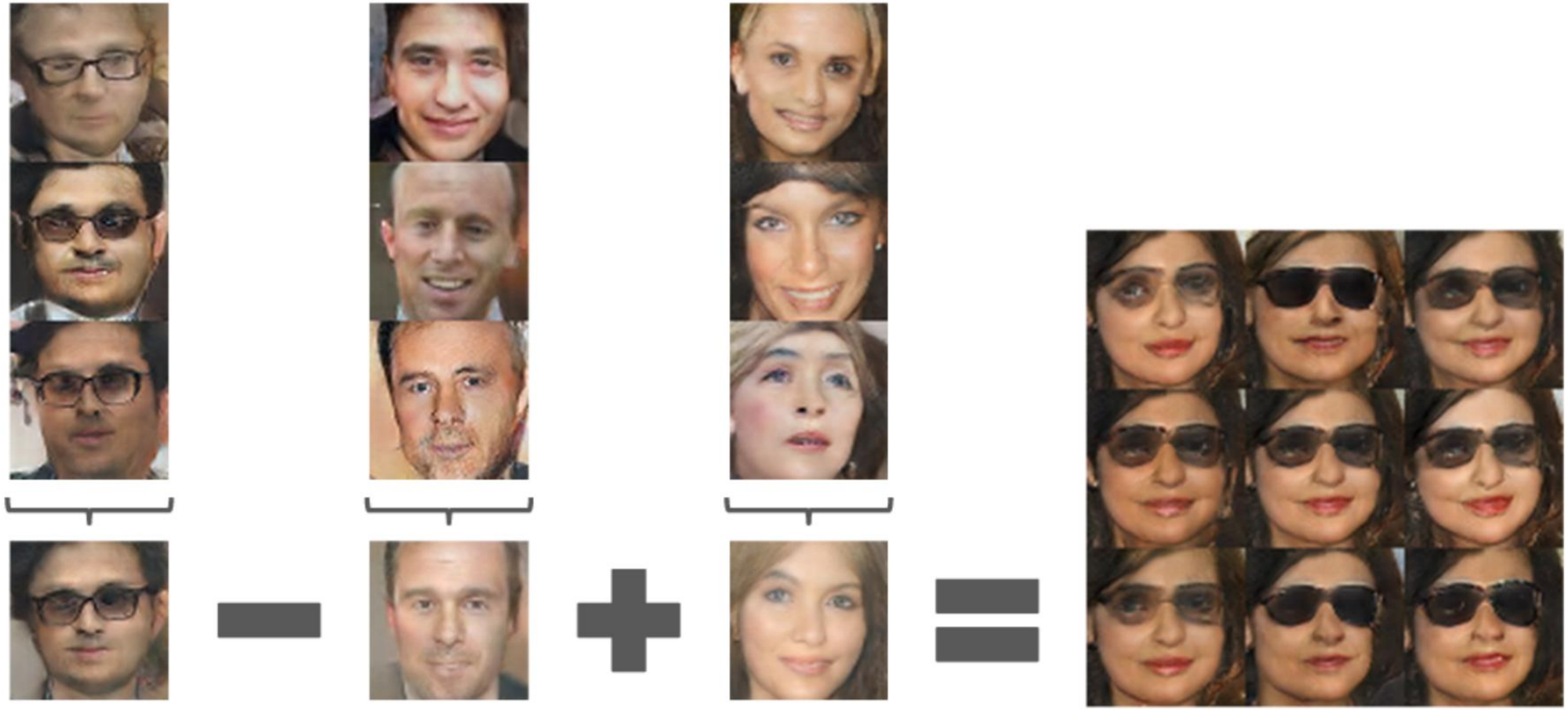
TRAINING GENERATOR



HOW GANS ARE BEING USED?

- Applied for modelling natural images.
- Performance is fairly good in comparison to other generative models.
- Useful for unsupervised learning tasks.




man
with glasses
man
without glasses
woman
without glasses
woman with glasses

HOW TO TRAIN GANS?

- Objective of generative network - increase the error rate of the discriminative network.
- Objective of discriminative network – decrease binary classification loss.
- Discriminator training - backprop from a binary classification loss.
- Generator training - backprop the negation of the binary classification loss of the discriminator.



LOSS FUNCTIONS

$$\mathcal{L}(\hat{x}) = \min_{x \in data} (x - \hat{x})^2$$

Generator

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$$

Discriminator



TYPES OF GAN

- **1. Vanilla GAN**
  - Vanilla GAN is the simplest type of GAN. It consists of:
  - A generator and a discriminator both are built using multi-layer perceptrons (MLPs).
  - The model optimizes its mathematical formulation using stochastic gradient descent (SGD).
  - While foundational, Vanilla GAN can face problems like:
  - **Mode collapse**: The generator produces limited types of outputs repeatedly.
  - **Unstable training**: The generator and discriminator may not improve smoothly.



TYPES OF GAN

- **2. Conditional GAN (CGAN)**
  - Conditional GAN (CGAN) adds an additional conditional parameter to guide the generation process. Instead of generating data randomly they allow the model to produce specific types of outputs.
  - Working of CGANs:
  - A conditional variable (y) is fed into both the generator and the discriminator.
  - This ensures that the generator creates data corresponding to the given condition (e.g generating images of specific objects).
  - The discriminator also receives the labels to help distinguish between real and fake data.
  - **Example**: Instead of generating any random image, CGAN can generate a specific object like a dog or a cat based on the label.



TYPES OF GAN

- **3. Deep Convolutional GAN (DCGAN)**
  - Deep Convolutional GAN (DCGAN) are among the most popular types of GANs used for image generation.
  - They are important because they:
  - Uses Convolutional Neural Networks (CNNs) instead of simple multi-layer perceptrons (MLPs).
  - Max pooling layers are replaced with convolutional stride helps in making the model more efficient.
  - Fully connected layers are removed, which allows for better spatial understanding of images.
  - DCGANs are successful because they generate high-quality, realistic images.



TYPES OF GAN

- **4. Laplacian Pyramid GAN (LAPGAN)**
  - Laplacian Pyramid GAN (LAPGAN) is designed to generate ultra-high-quality images by using a multi-resolution approach.
  - Working of LAPGAN:
  - Uses multiple generator-discriminator pairs at different levels of the Laplacian pyramid.
  - Images are first down sampled at each layer of the pyramid and upscaled again using Conditional GAN (CGAN).
  - This process allows the image to gradually refine details and helps in reducing noise and improving clarity.
  - Due to its ability to generate highly detailed images, LAPGAN is considered a superior approach for photorealistic image generation.



TYPES OF GAN

- **5. Super Resolution GAN (SRGAN)**
  - Super-Resolution GAN (SRGAN) is designed to increase the resolution of low-quality images while preserving details.
  - Working of SRGAN:
  - Uses a deep neural network combined with an adversarial loss function.
  - Enhances low-resolution images by adding finer details helps in making them appear sharper and more realistic.
  - Helps to reduce common image upscaling errors such as blurriness and pixelation.



MAJOR DIFFICULTIES

- Networks are difficult to converge.
- Ideal goal – Generator and discriminator to reach some desired equilibrium but this is rare.
- GANs are yet to converge on large problems (E.g. Imagenet).



COMMON FAILURE CASES

- The discriminator becomes too strong too quickly and the generator ends up not learning anything.
- The generator only learns very specific weaknesses of the discriminator.
- The generator learns only a very small subset of the true data distribution.



SO WHAT CAN WE DO?

- Normalize the inputs
- A modified loss function
- BatchNorm
- Avoid Sparse Gradients: ReLU, MaxPool
- Use Soft and Noisy Labels
- DCGAN / Hybrid Models
- Track failures early (D loss goes to 0: failure mode)
- If you have labels, use them
- Add noise to inputs, decay over time



APPLICATIONS OF GANS

- **Image Synthesis & Generation:**
  - Generate realistic images, avatars and high-resolution visuals by learning patterns from training data.
  - Used in art, gaming and AI-driven design.
- **Image-to-Image Translation:**
  - Transform images between domains while preserving key features.
  - Examples include converting day images to night, sketches to realistic images or changing artistic styles.
- **Text-to-Image Synthesis:**
  - Create visuals from textual descriptions helps applications in AI-generated art, automated design and content creation.

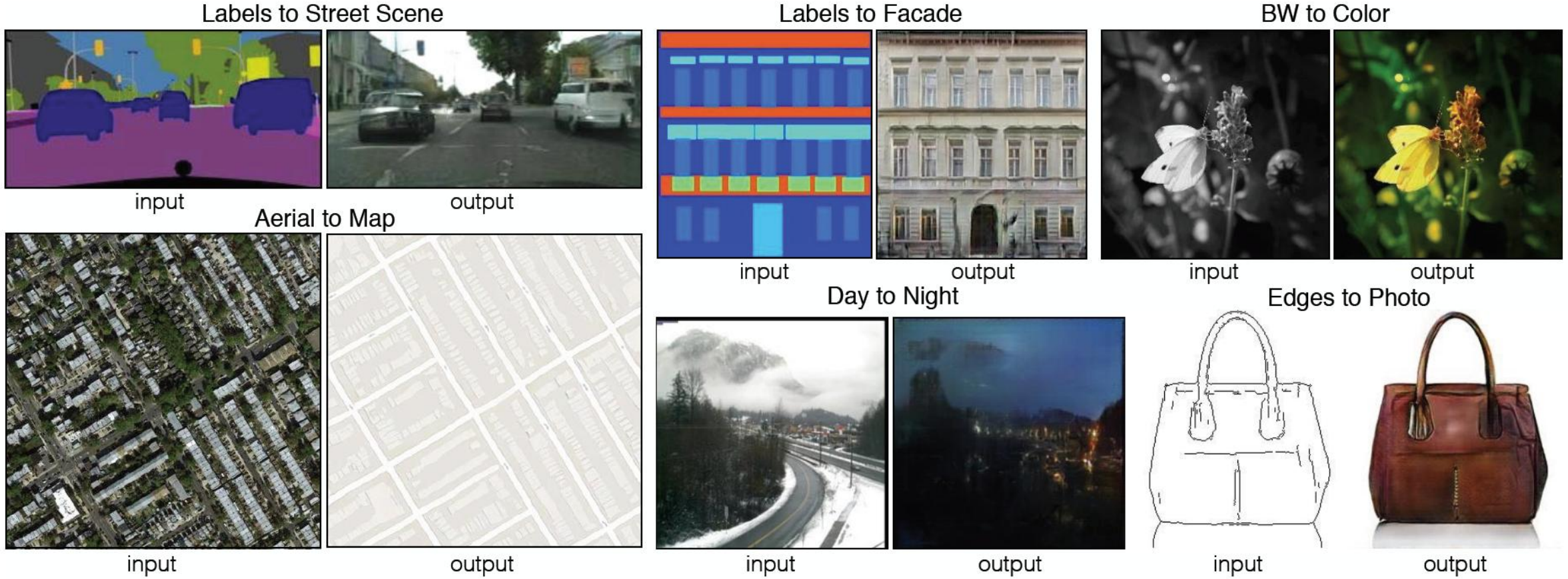


APPLICATIONS OF GANS

- **Data Augmentation:**
  - Generate synthetic data to improve machine learning models helps in making them more robust and generalizable in fields with limited labeled data.
- **High-Resolution Image Enhancement:**
  - Upscale low-resolution images which helps in improving clarity for applications like medical imaging, satellite imagery and video enhancement.



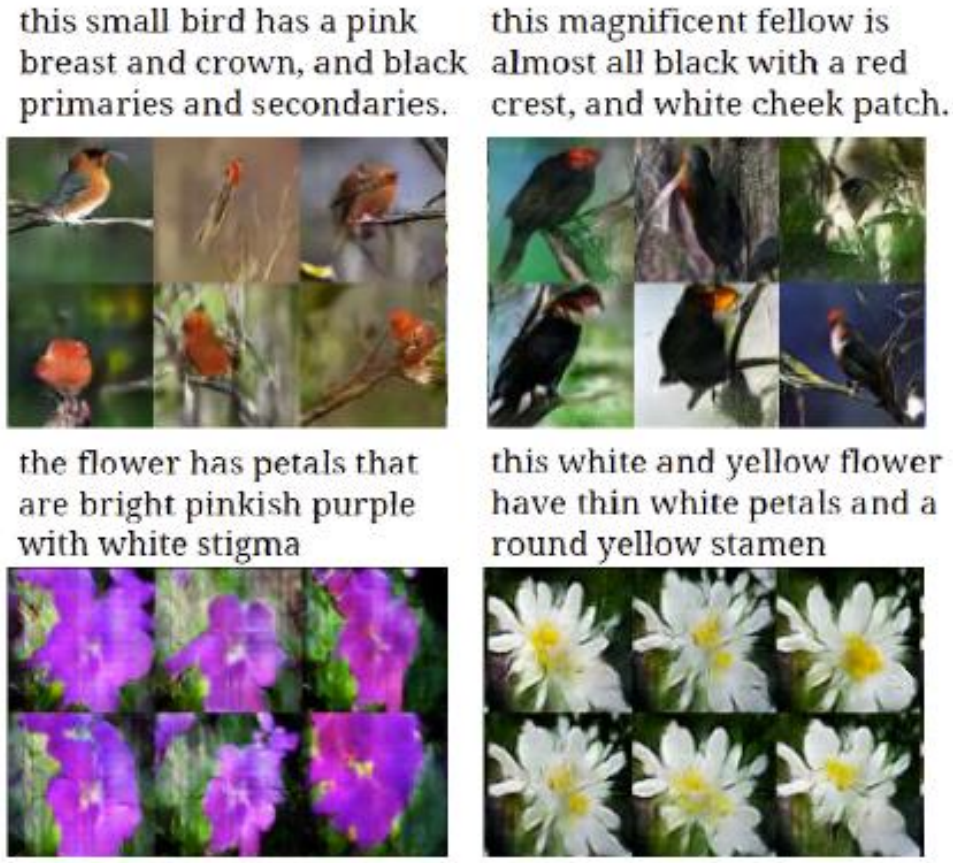
APPLICATIONS OF GANS



Isola, P., Zhu, J. Y., Zhou, T., & Efros, A. A. "**Image-to-image translation with conditional adversarial networks**". arXiv preprint arXiv:1611.07004. (2016).

APPLICATIONS OF GANS

- Motivation
- Given a text description, generate images closely associated.
- Uses a conditional GAN with the generator and discriminator being condition on “dense” text embedding.



WHY USE GANS FOR GENERATION?

- Can be trained using back-propagation for Neural Network based Generator/Discriminator functions.
- Sharper images can be generated.
- Faster to sample from the model distribution: *single* forward pass generates a *single* sample.



REFERENCES

- https://tryolabs.com/blog/2016/12/06/major-advancements-deep-learning-2016/
- https://blog.waya.ai/introduction-to-gans-a-boxing-match-b-w-neural-nets-b4e5319cc935#.6l7zh8u50
- https://en.wikipedia.org/wiki/Generative_adversarial_networks
- http://blog.aylien.com/introduction-generative-adversarial-networks-code-tensorflow/
- https://github.com/soumith/ganhacks
- https://www.youtube.com/watch?v=i64KpxyaLpo
- https://www.youtube.com/watch?v=RAa55G-oEuk
- https://www.cs.toronto.edu/~hinton/digitmovie2.gif

